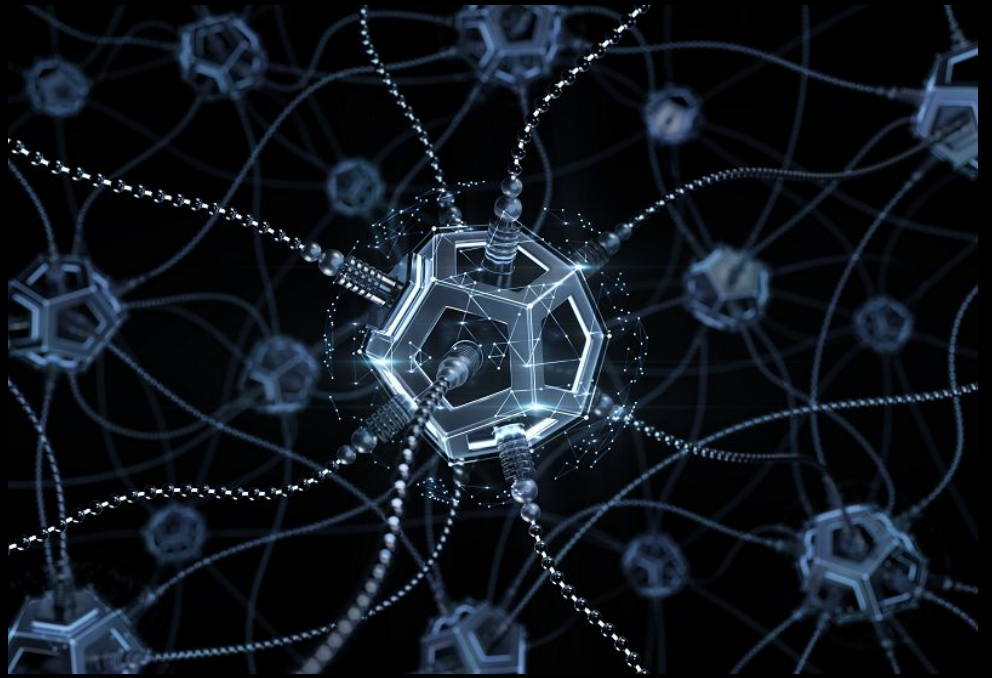
PUCP

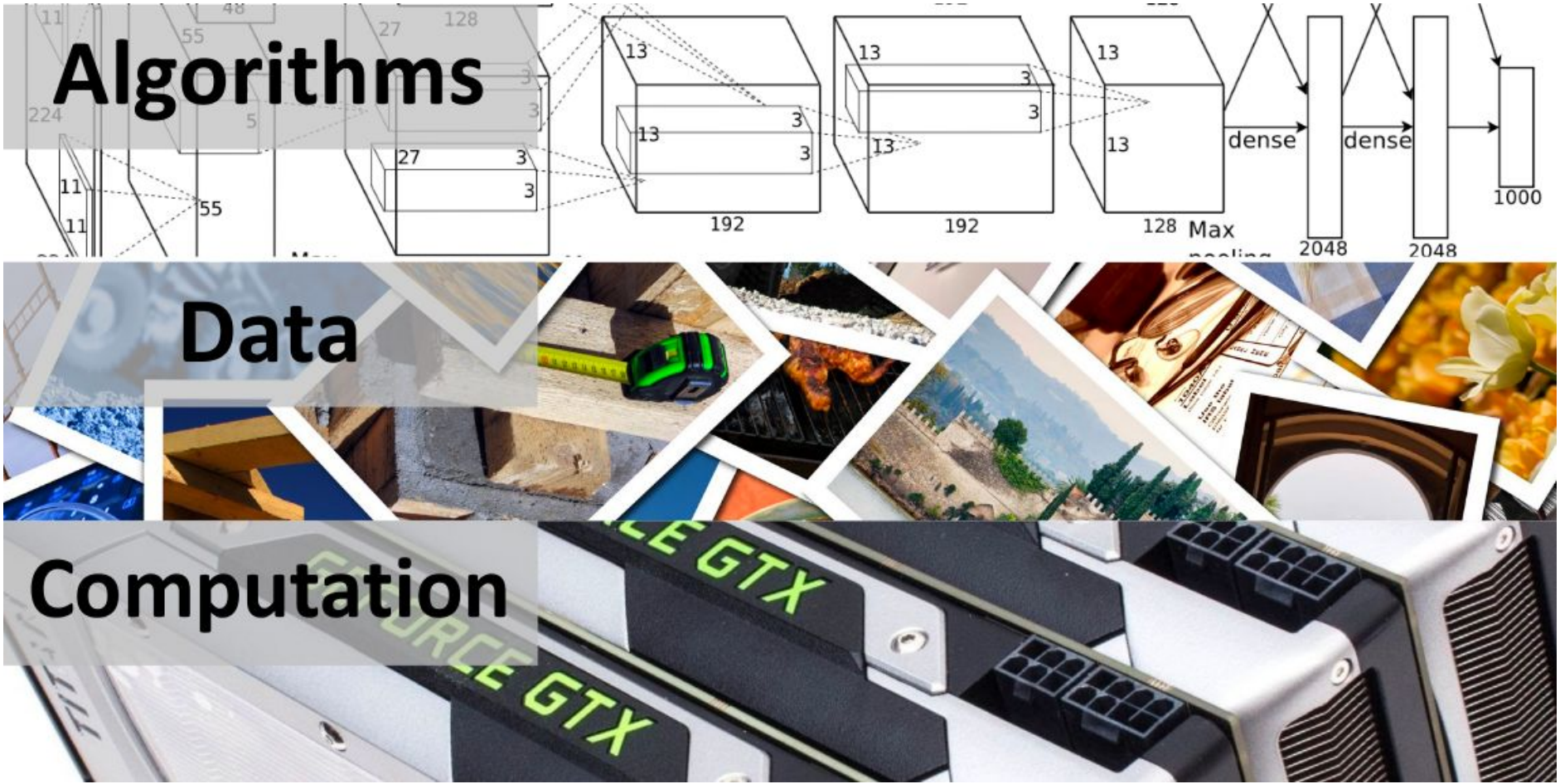
MAESTRÍA EN INFORMÁTICA

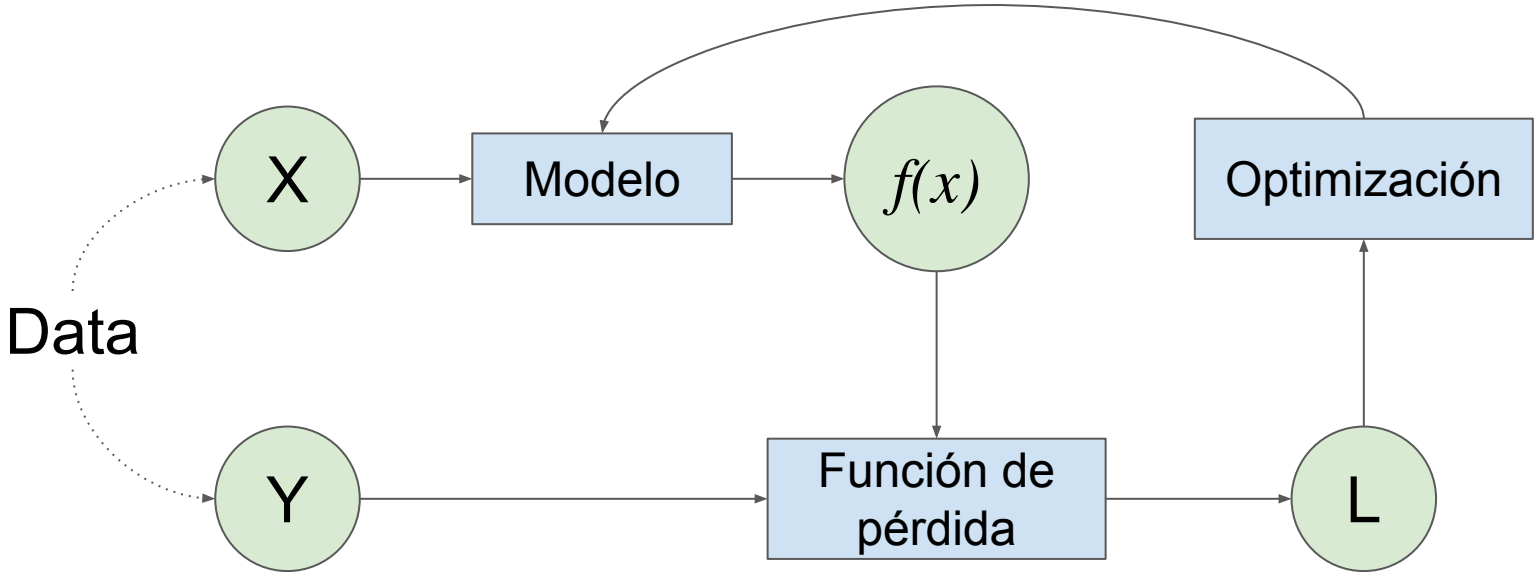
Deep Learning



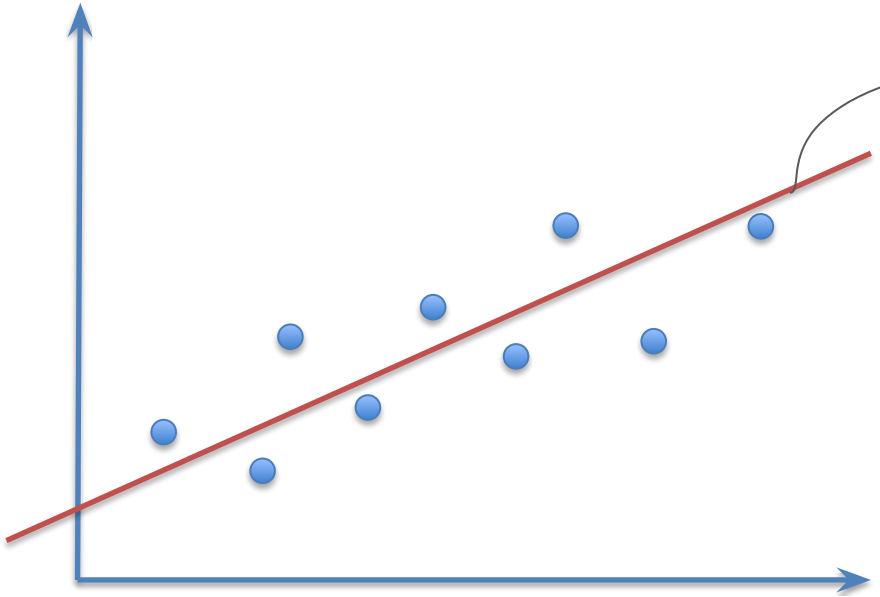
Tema 3: Fundamentos de Redes Neuronales Artificiales

Prof. Dr. César A. Beltrán Castañón


Algorithms
Data
Computation
dense
dense
1000
2048
2048
192
192
128
Max


Data
X
Y
Modelo
$f(x)$
Función de pérdida
L
Optimización

Regresión lineal simple

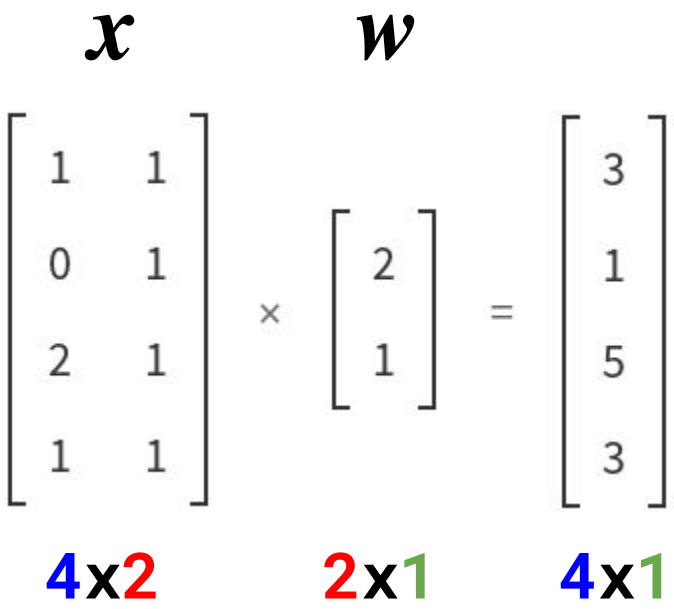


$$f(x) = ax + b$$

Donde:

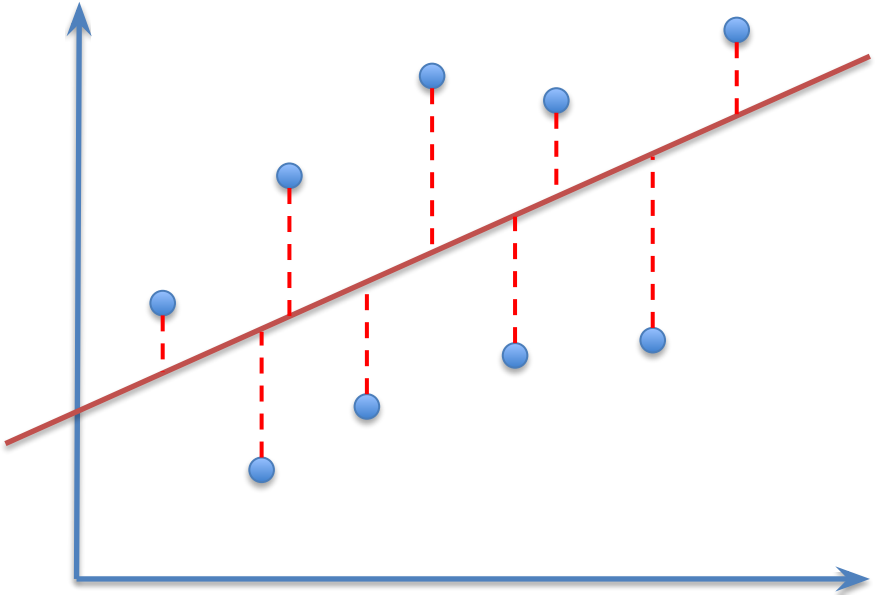
a = pendiente

b = término de intercepción (bias)



http://matrixmultiplication.xyz

Función de pérdida: MSE (Mean squared error)



Mean squared error:

$$L = \frac{1}{2m} \sum_{i=1}^{m} (f(x_i) - y_i)^2$$

Data x

Modelo f(x)

Error f(x) - y

Optimización: Gradient descent

$$f(x_i) = ax_i + b$$

$$L = \frac{1}{2m}\sum_{i=1}^{m}(f(x_i) - y_i)^2$$

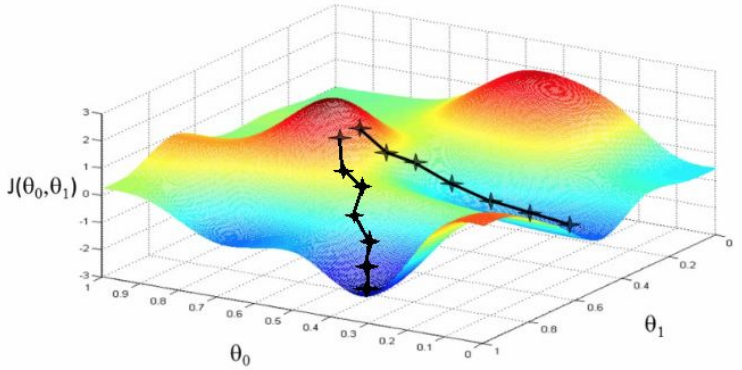
$$\frac{dL}{da} = \sum_{i=1}^{m}(f(x_i) - y_i)(x_i)$$

$$\frac{dL}{db} = \sum_{i=1}^{m}(f(x_i) - y_i)$$

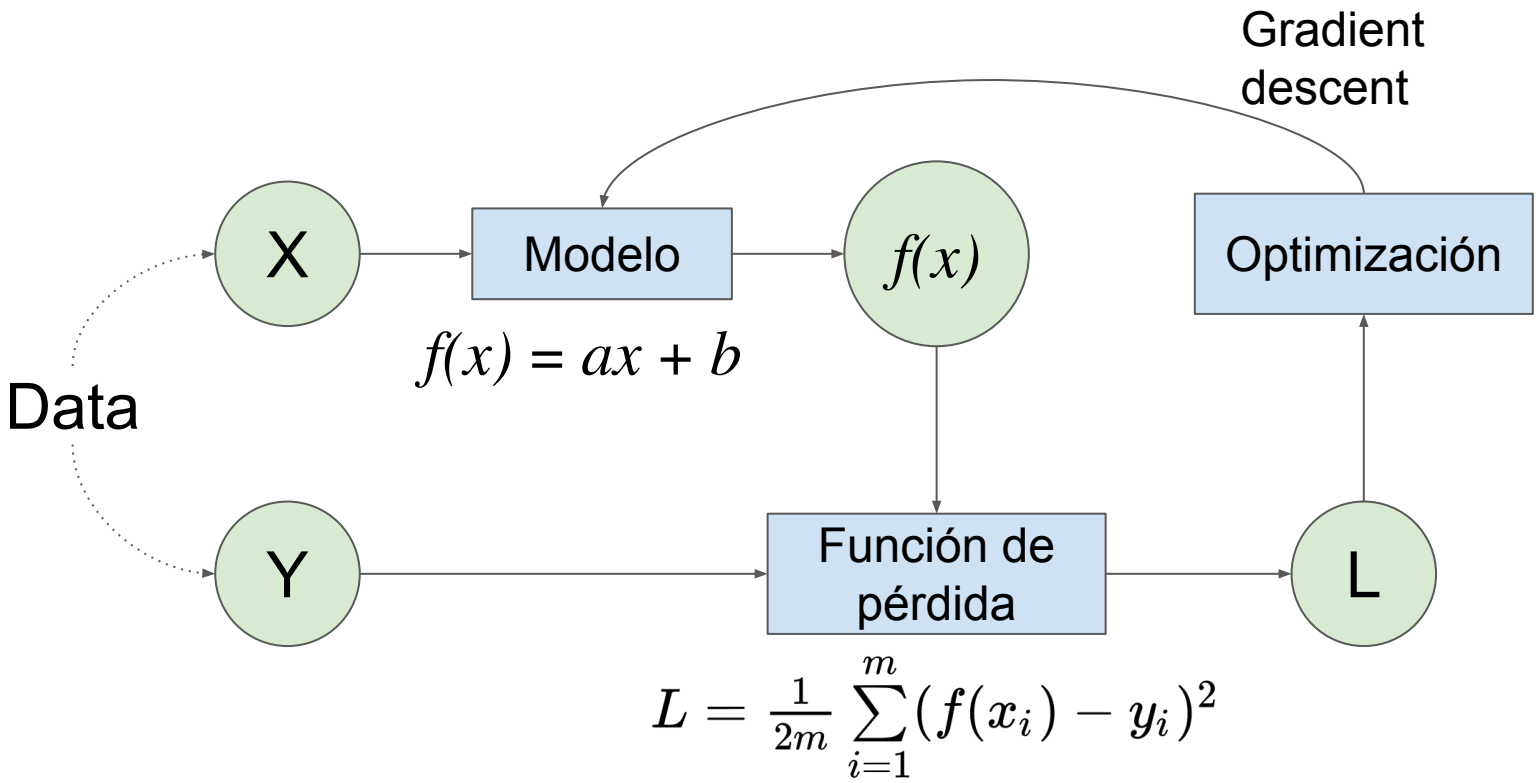
$$iterar:$$

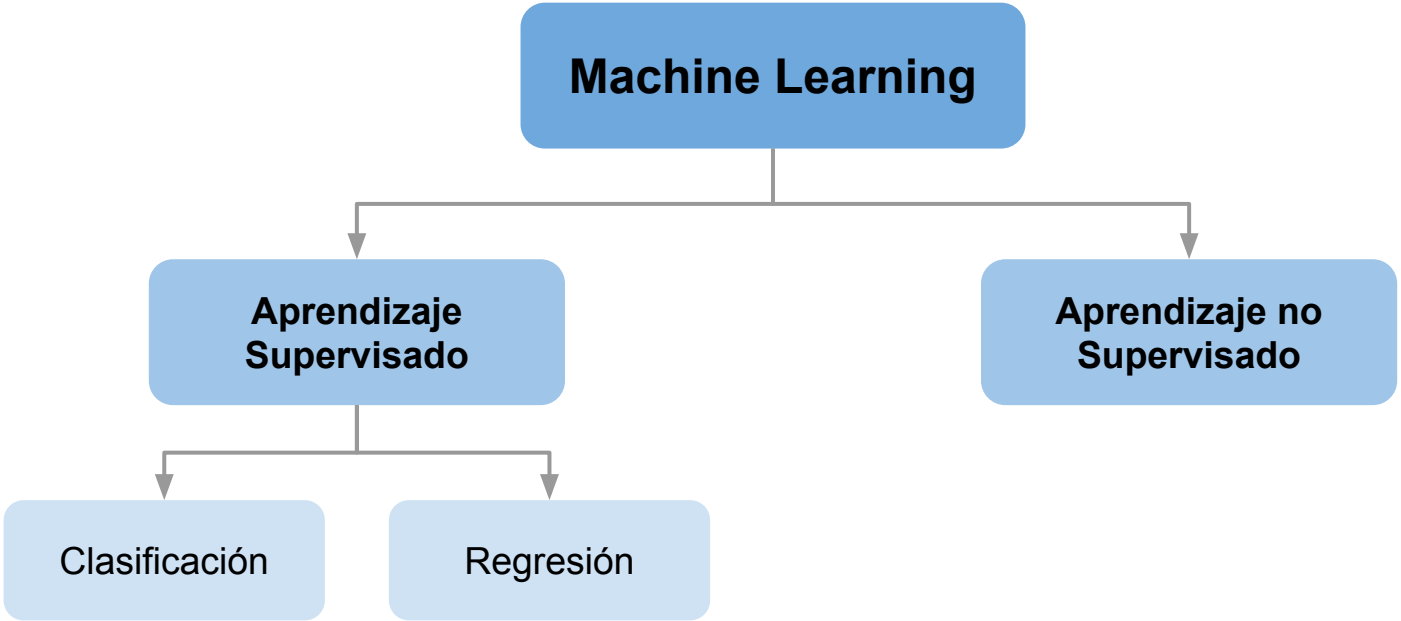
$$new\, a = a - lr * (\frac{dL}{da})$$

$$new\, b = b - lr * (\frac{dL}{db})$$

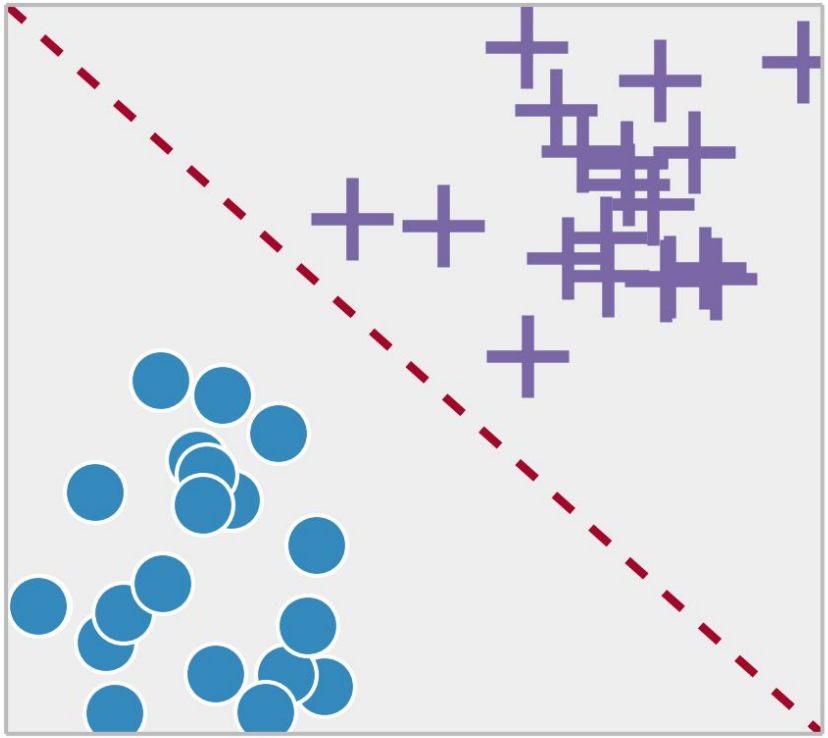


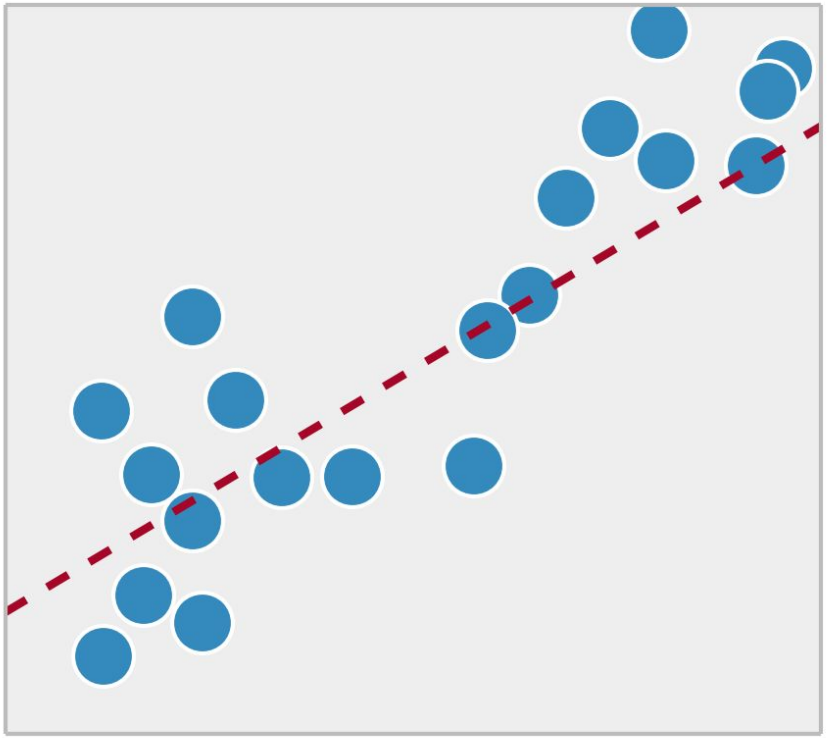
Regresión lineal simple




Machine Learning
Aprendizaje Supervisado
Aprendizaje no Supervisado
Clasificación
Regresión

Clasificación vs Regresión

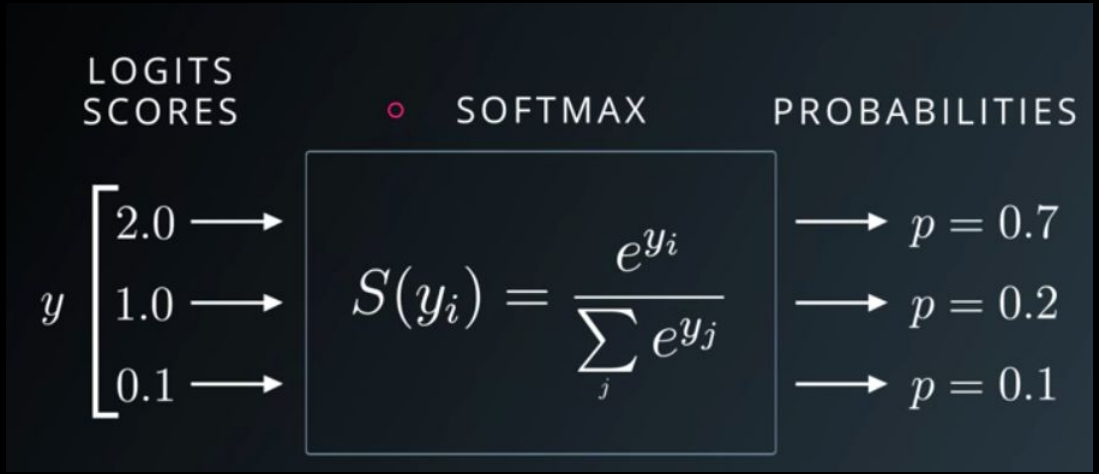




PUCP

MAESTRIA EN INFORMÁTICA

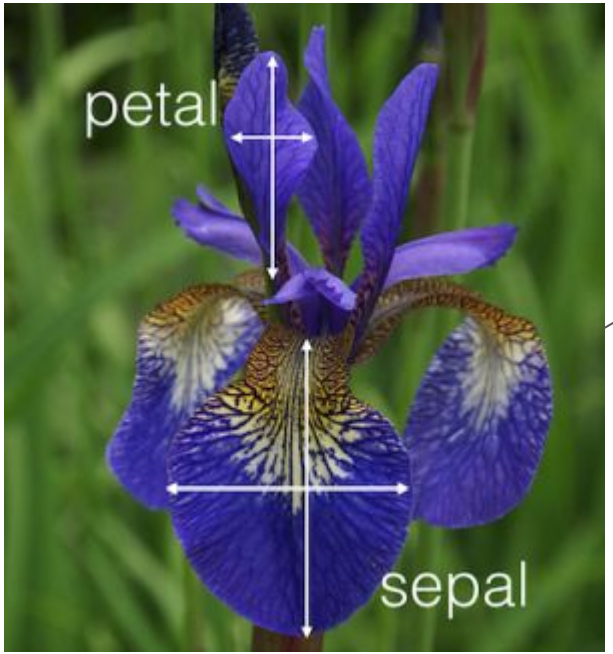
Deep Learning



Tema 3.1: Regresión Logística Multinomial (Softmax)

Prof. Dr. César A. Beltrán Castañón

Iris dataset



sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2
5.7	4.4	1.5	0.4
6.7	3.1	4.4	1.4
4.8	3.4	1.6	0.2
4.4	3.2	1.3	0.2

Target: class of iris plant

Iris dataset



Iris setosa



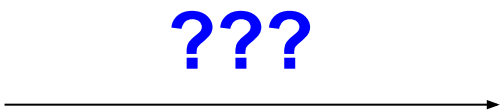
Iris versicolor



Iris virginica

Objetivo

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2



setosa!

~~versicolor~~

~~virginica~~

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

???

setosa!

Variables independientes:
[4x1]

3 resultados, indicando la probabilidad de cada clase:

[1, 0, 0]
Setosa Versicolor Virginica
(one hot encoding)

[3x1]

Definición del modelo

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

data weights

f(**x**,**W**)

setosa!

Variables independientes:
[4x1]

3 resultados, indicando la probabilidad de cada clase:

[1, 0, 0]

Setosa Versicolor Virginica

(one hot encoding)

[3x1]

Definición del modelo: **Clasificador Lineal**

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

Variables independientes:
[4x1]

data weights

f(**x**,**W**)

$$f(x, W) = Wx$$

setosa!

3 resultados, indicando la probabilidad de cada clase:

[1, 0, 0]

Setosa Versicolor Virginica

(one hot encoding)

[3x1]

Definición del modelo: **Clasificador Lineal**

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2



setosa!

$$f(x, W) = Wx$$

4xN

3xN

?

parameters, or “weights”

Definición del modelo: **Clasificador Lineal**

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2



setosa!

$$f(x, W) = Wx$$

4xN

3xN

[3x4]

parameters, or “weights”

Definición del modelo: **Clasificador Lineal**

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

→ **setosa!**

$$f(x, W) = Wx + b$$

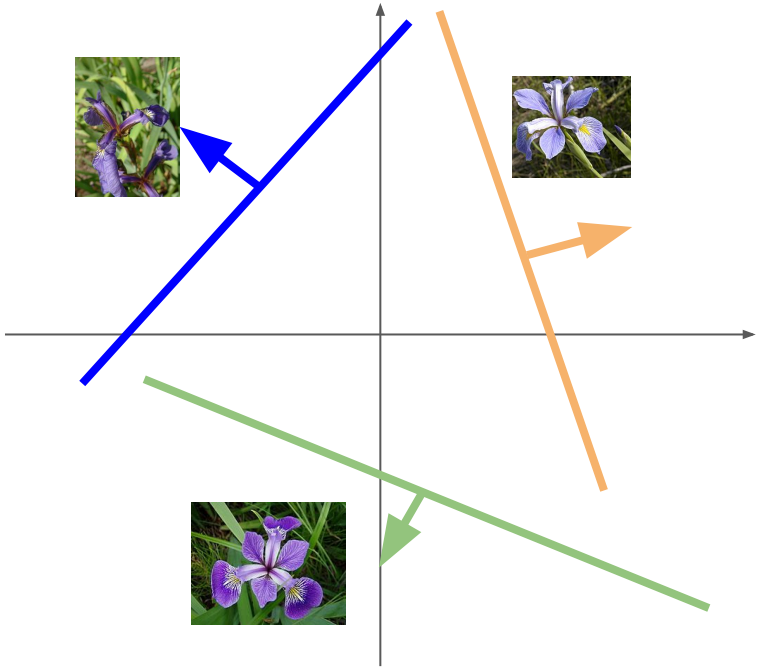
3xN = [3x4] · 4xN + (+b) 3x1

bias

parameters, or “weights”

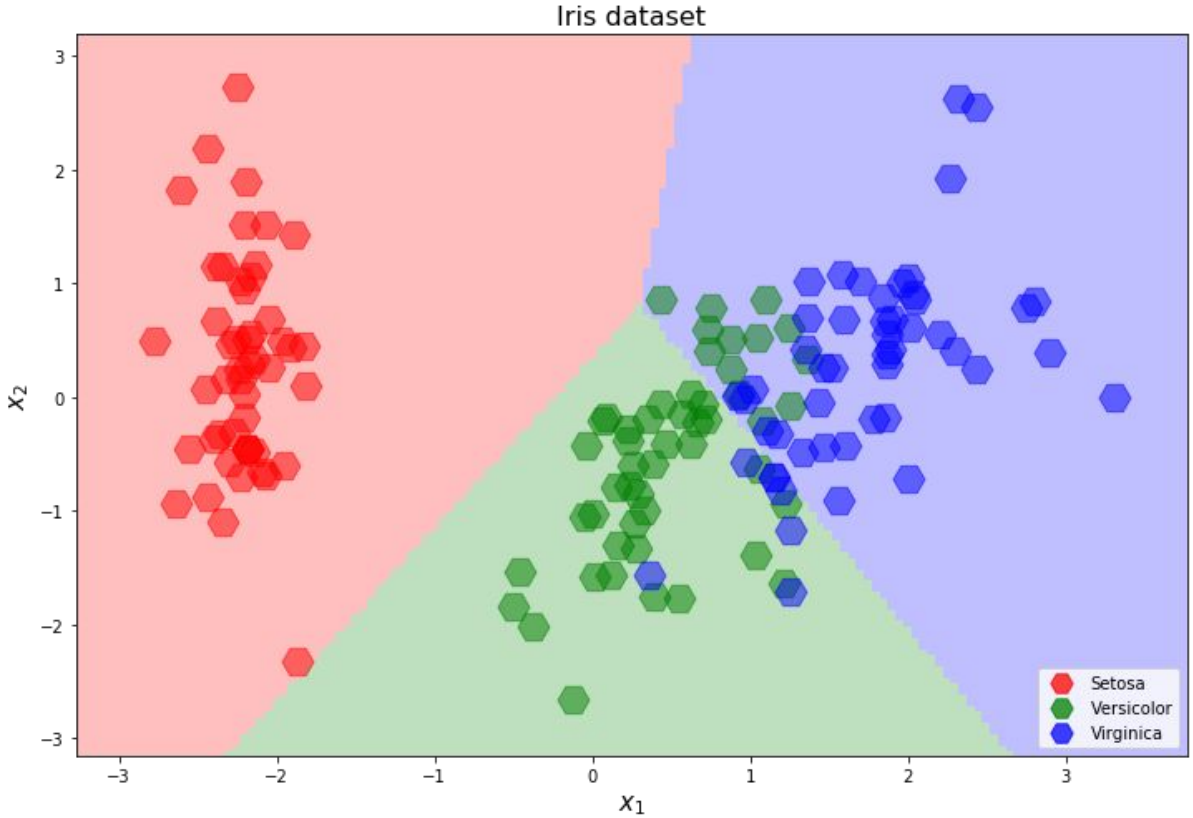
Interpretando el clasificador lineal

$$f(x_i) = Wx_i + b$$



Setosa
Versicolor
Virginica

Interpretando el clasificador lineal



Ej:

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

setosa!

[1, 0, 0]

Setosa Versicolor Virginica

4.6
3.6
1.0
0.2

X_i

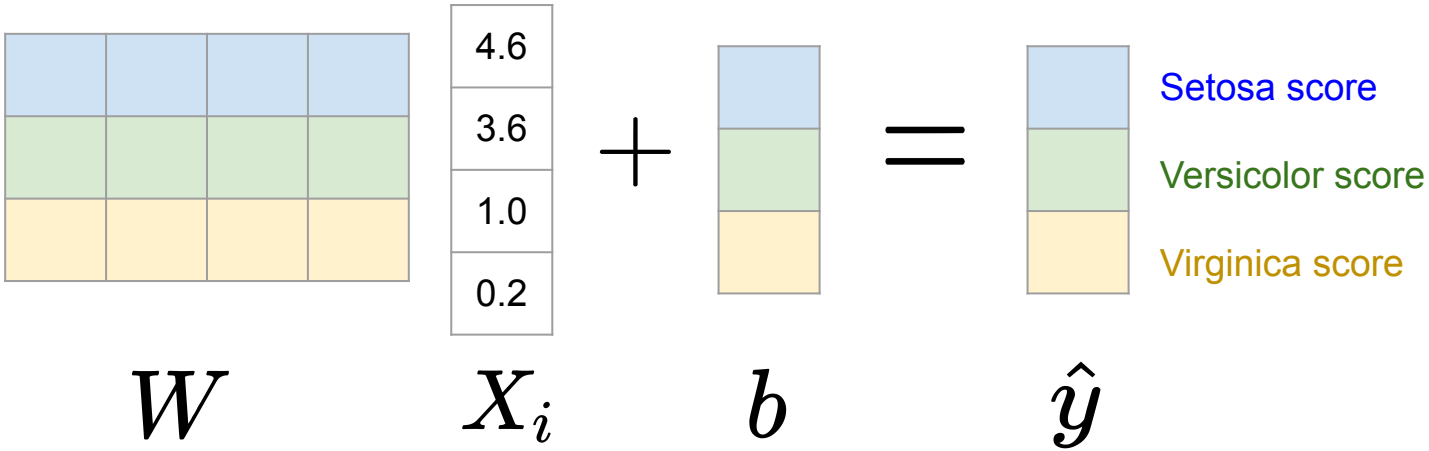
1
0
0

y_i

Ej:

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

$$\hat{y} = f(x_i) = Wx_i + b$$



Ej:

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

Random numbers!

$$\hat{y} = f(x_i) = Wx_i + b$$

-0.4	-0.1	-2.1	1.6	4.6	+	1.8	=	-2.18
-1.8	-0.8	0.5	-1.2	3.6		0.4		-10.5
-1.1	-0.9	0.6	2.3	1.0		0.1		-7.14
				0.2				
W				X_i		b		$\hat{y}$

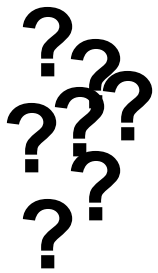
Setosa score

Versicolor score

Virginica score

¿Cómo interpretamos estos scores?

-2.18	Setosa score
-10.5	Versicolor score
-7.14	Virginica score



Softmax Classifier (Multinomial Logistic Regression)

-2.18	Setosa score
-10.5	Versicolor score
-7.14	Virginica score

$$softmax(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$

Softmax Classifier (Multinomial Logistic Regression)

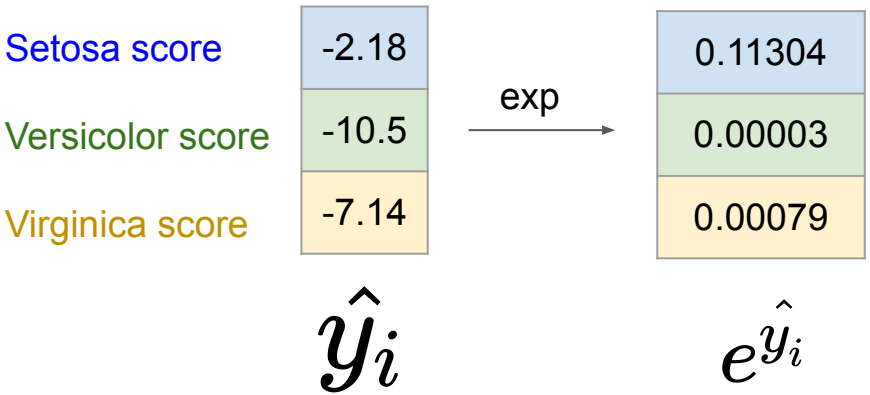
$$softmax(\hat{y}_i) = \frac{e^{\hat{y_i}}}{\sum_j e^{\hat{y_j}}}$$

Setosa score	-2.18
Versicolor score	-10.5
Virginica score	-7.14

$\hat{y}_i$

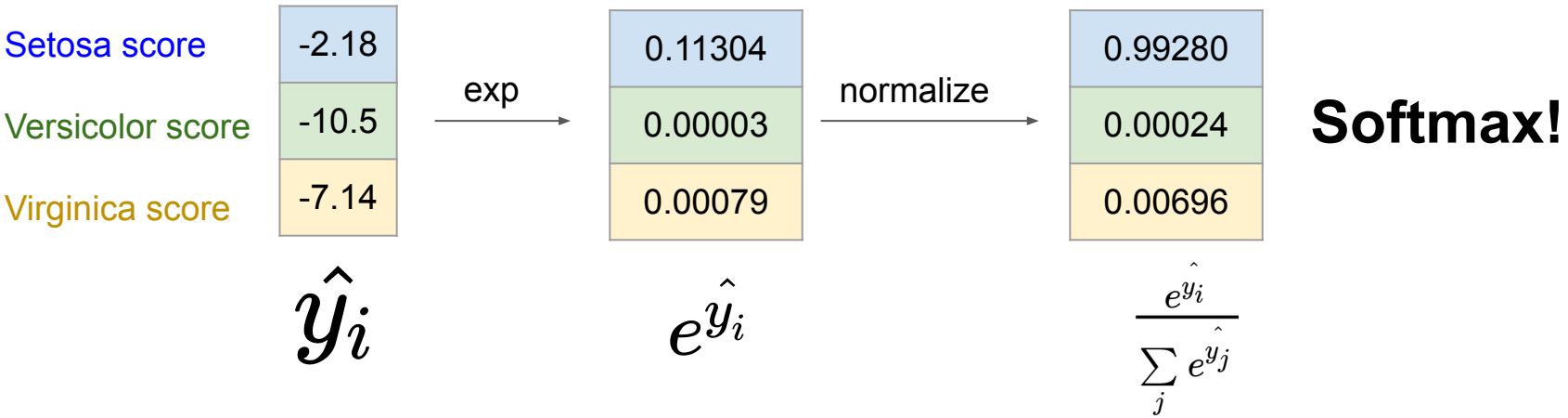
Softmax Classifier (Multinomial Logistic Regression)

$$softmax(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$



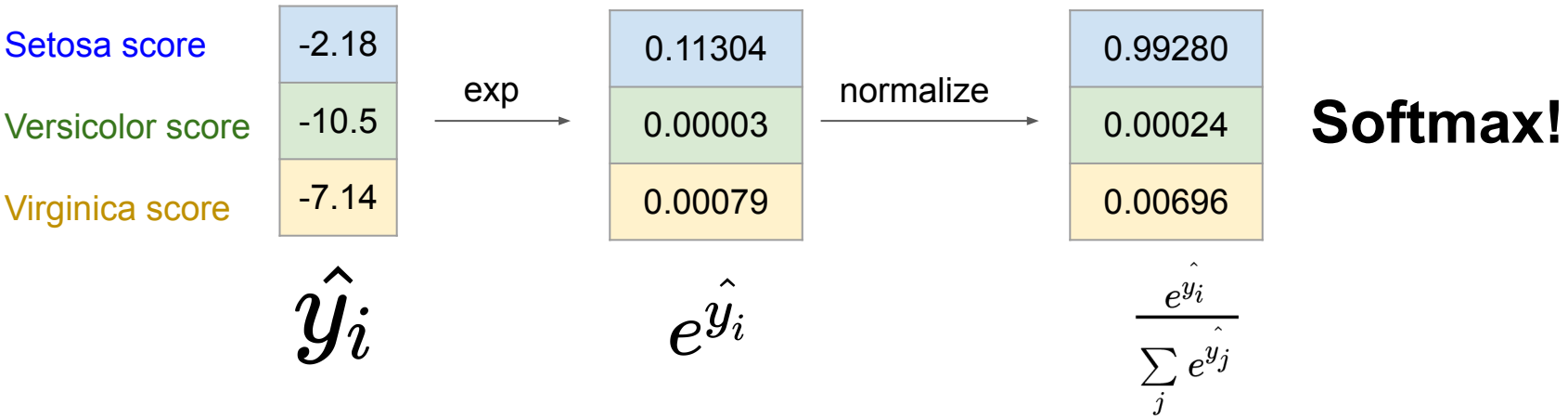
Softmax Classifier (Multinomial Logistic Regression)

$$softmax(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$

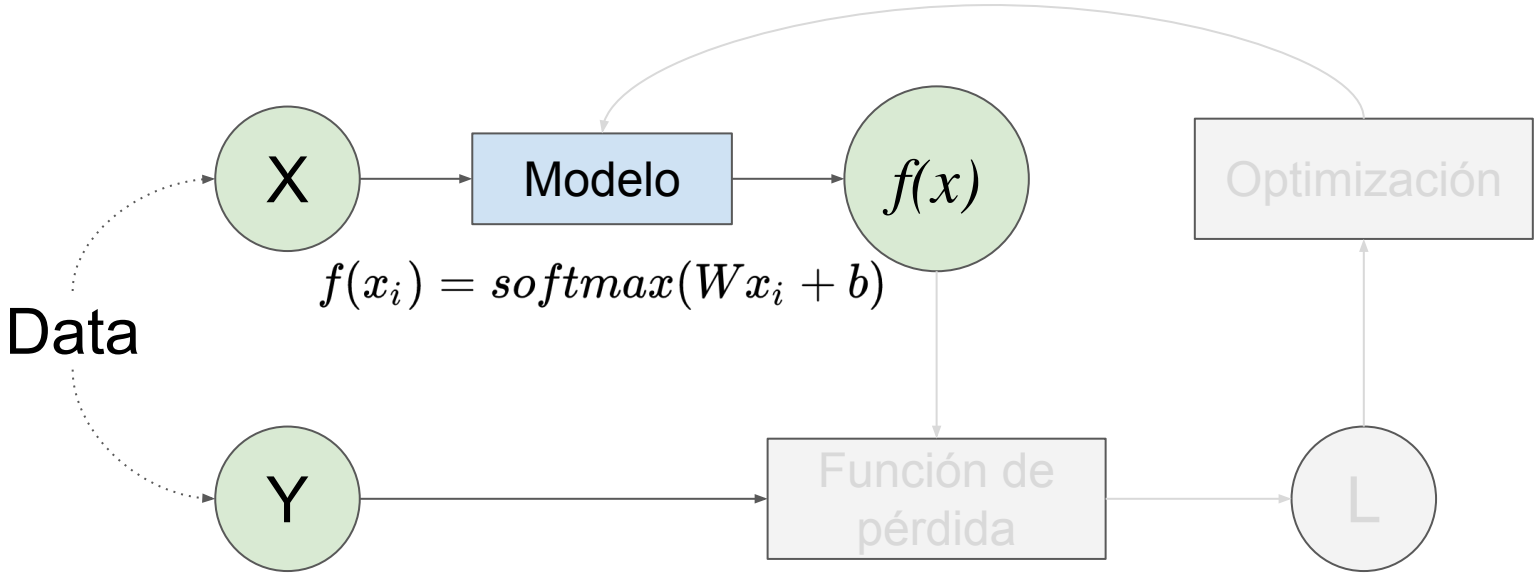


Definición final del modelo

$$f(x_i) = softmax(Wx_i + b)$$



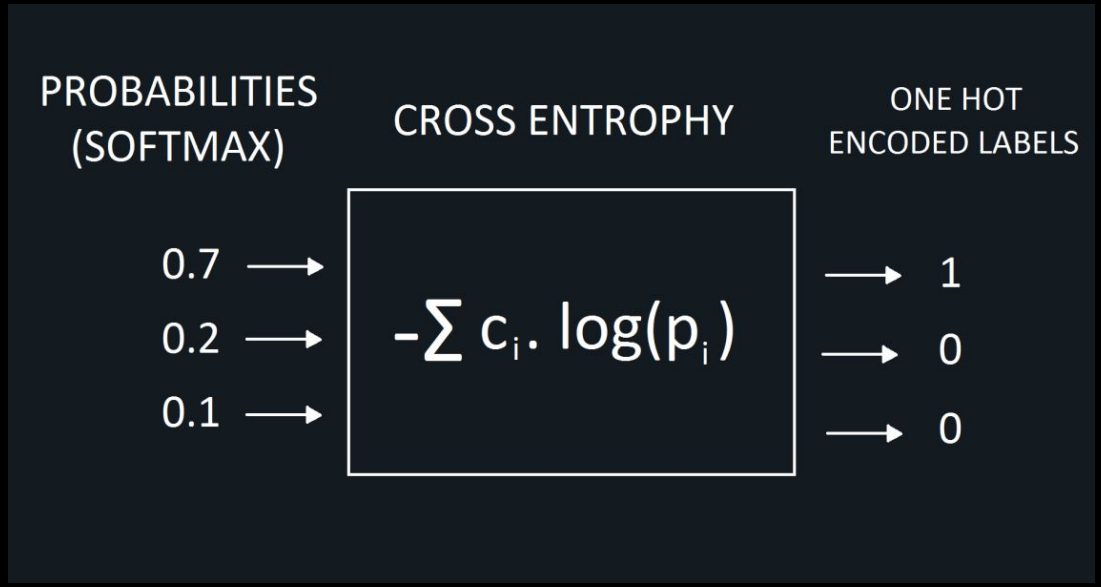
Regresión logística



PUCP

MAESTRÍA EN INFORMÁTICA

Deep Learning



Tema 3.2: Función de pérdida (Cross-Entropy)

Prof. Dr. César A. Beltrán Castañón

¿Cómo evaluamos el modelo?

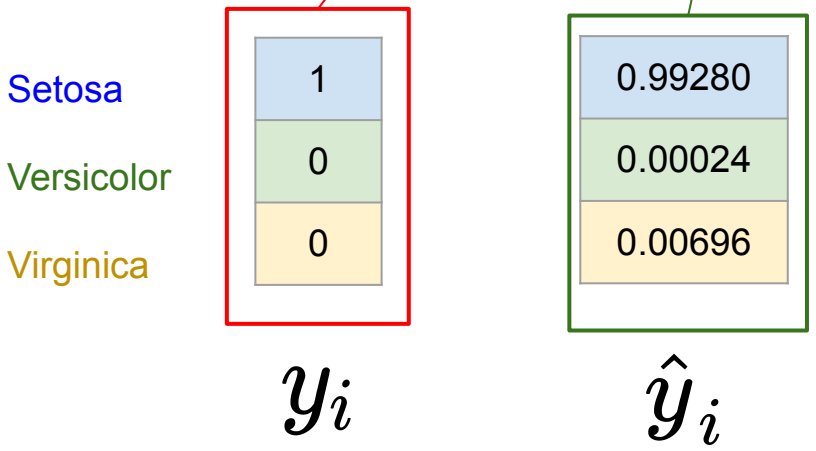
Cross Entropy (Función de pérdida)

$$L = -\sum p(x) \log q(x)$$

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i)$$

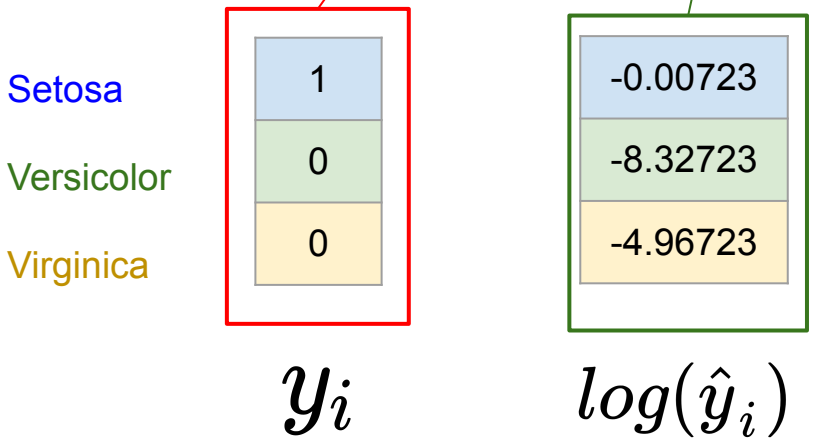
Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i)$$



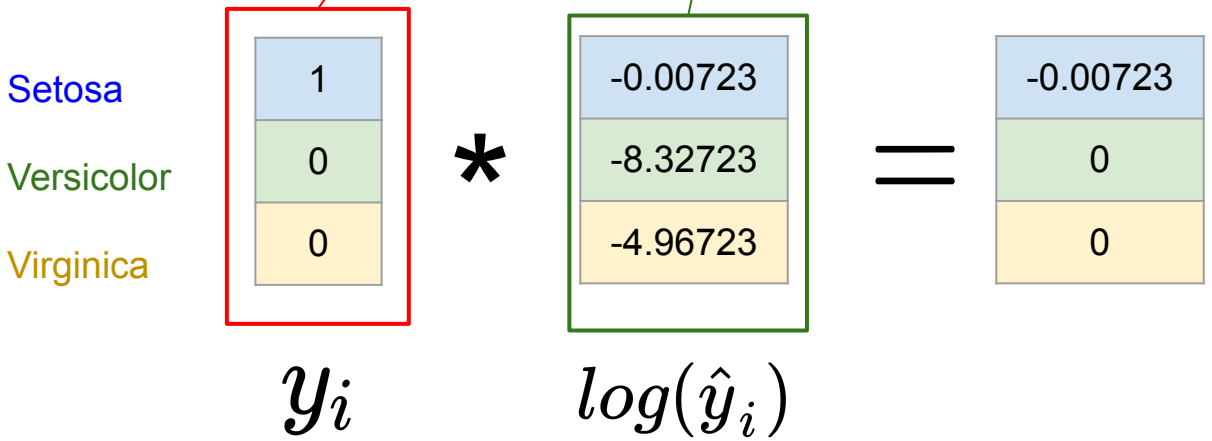
Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i \log(\hat{y}_i)$$



Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i \log(\hat{y}_i)$$



Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i)$$

$$L = -\sum \begin{array}{|c|}\hline -0.00723 \\ \hline 0 \\ \hline 0 \\ \hline \end{array} = \mathbf{0.00723}$$

Entropy

$$-\sum p(x) \log p(x)$$

Cross-entropy

$$-\sum p(x) \log q(x)$$

Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i) = -\sum$$

-0.00723
0
0

Setosa

Versicolor

Virginica

1
0
0

y_i

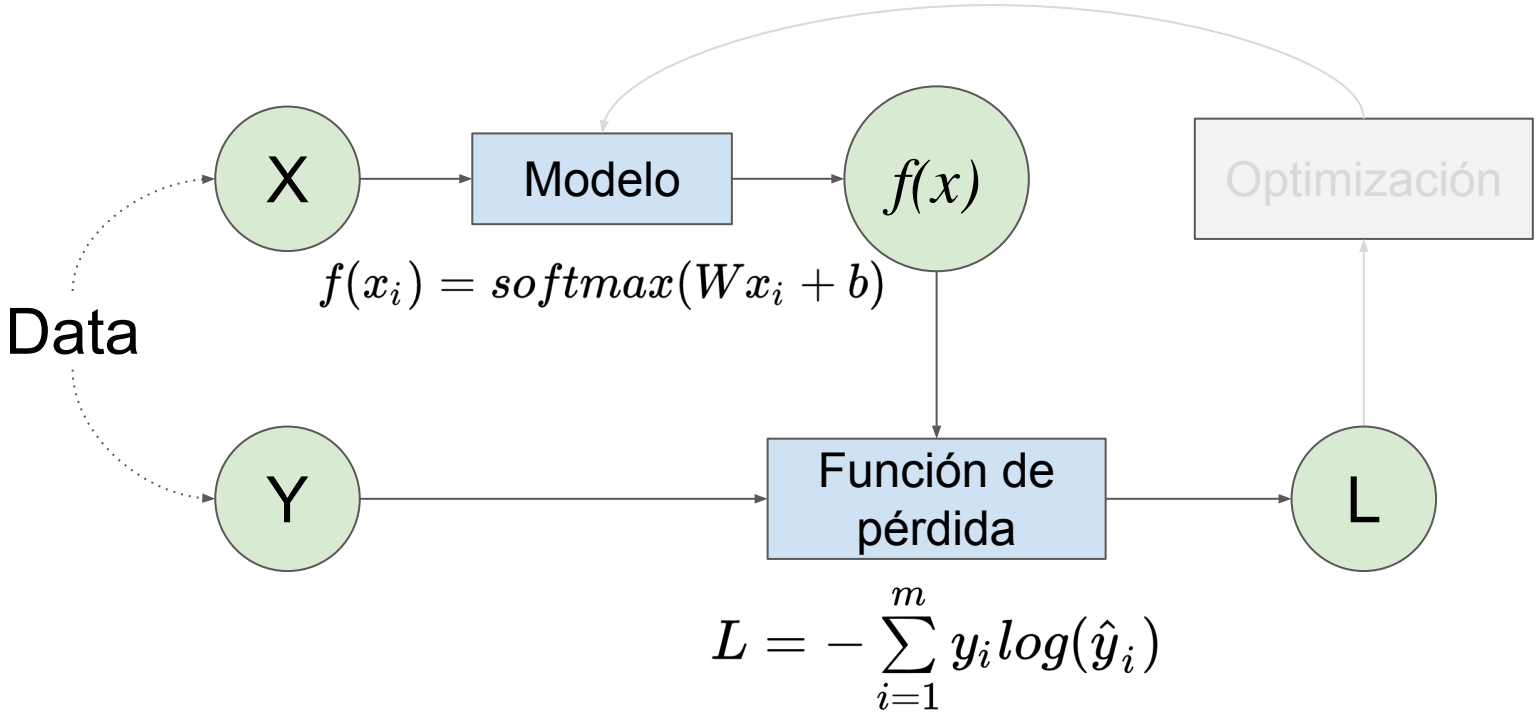
0.99280
0.00024
0.00696

$\hat{y}_i$

$= \mathbf{0.00723}$

Si usamos la predicción del modelo, **0.00723** representaría la información que obtendríamos al conocer la data real.

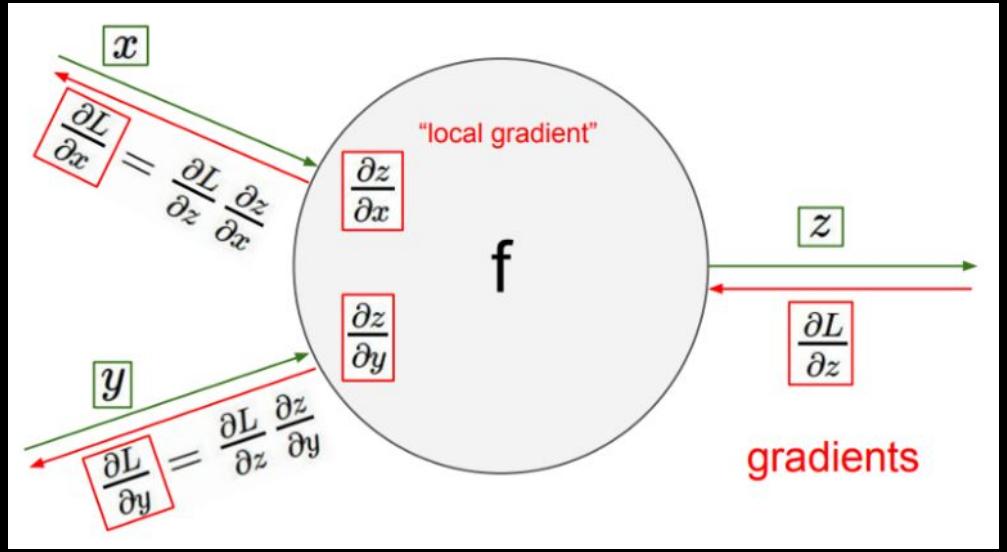
Regresión logística



PUCP

MAESTRIA EN INFORMÁTICA

Deep Learning



Tema 3.3: Optimización del modelo (Backpropagation)

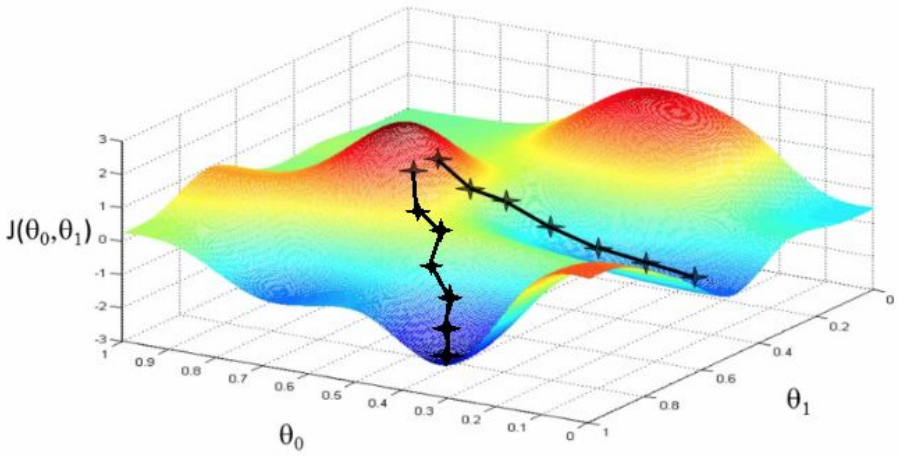
Prof. Dr. César A. Beltrán Castañón

Optimización

$$f(x_i) = softmax(Wx_i + b)$$

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i)$$

$$\frac{dL}{dW}, \frac{dL}{db} ?$$



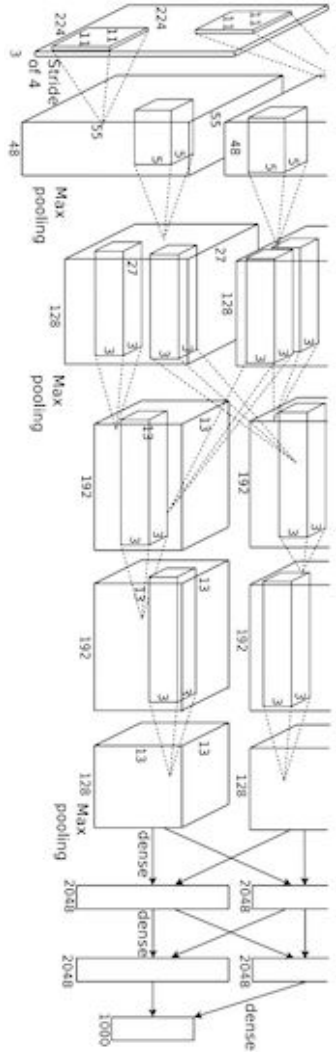
Convolutional Network (AlexNet)

input image

weights

loss

$$\frac{dL}{dW}?$$

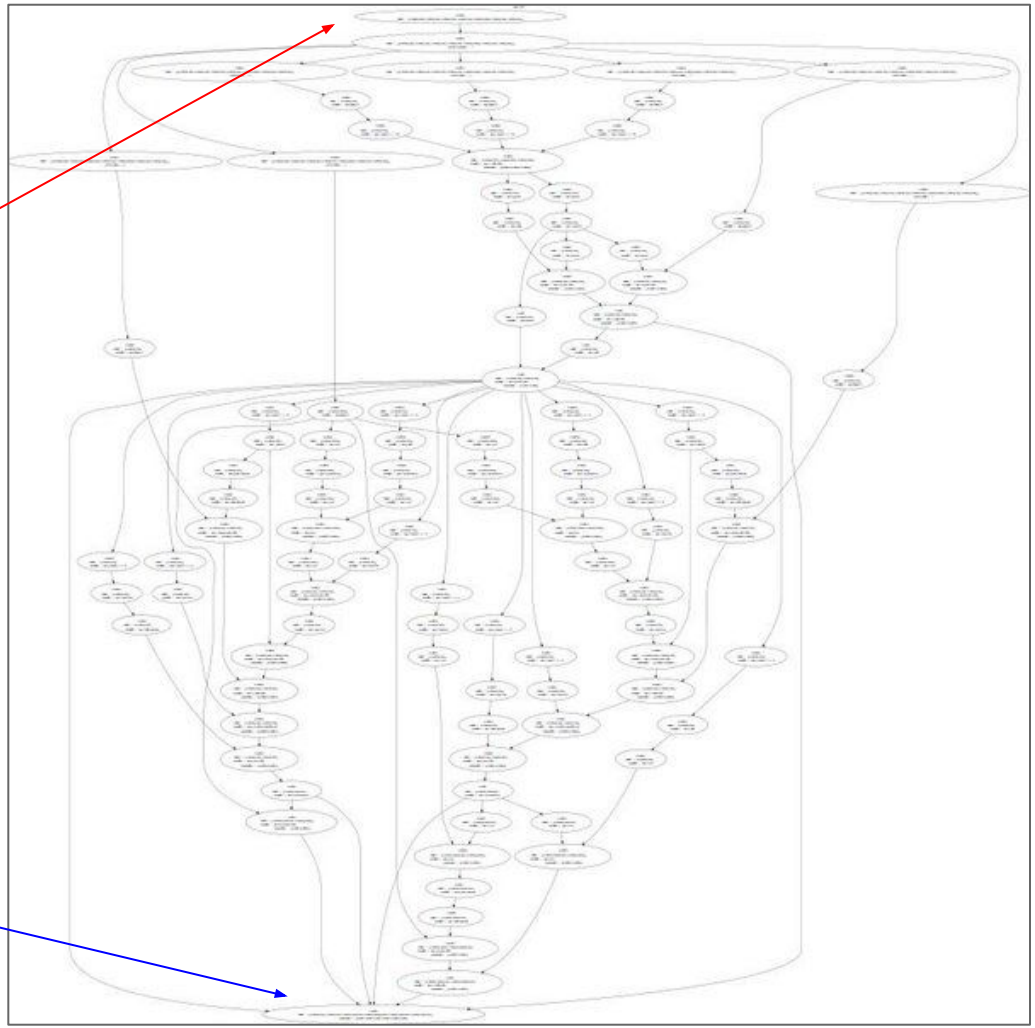


Neural Turing Machine

input tape

loss

$$\frac{dL}{dW}$$?

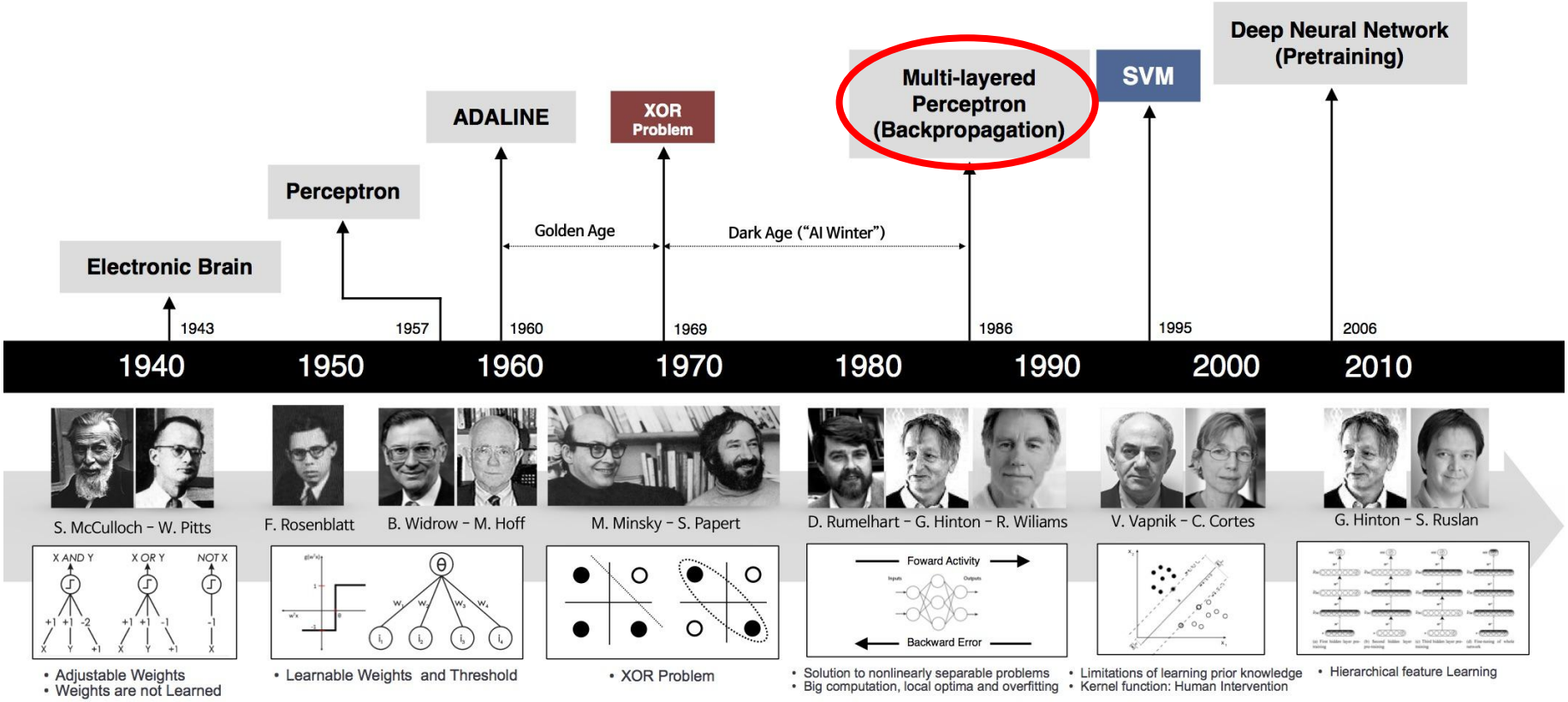


¿Cómo derivamos las gradientes?



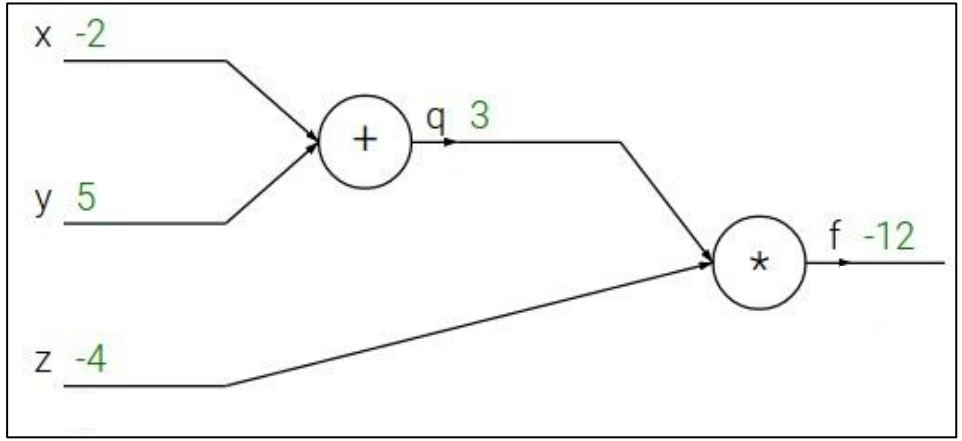
Backpropagation !




Deep Neural Network (Pretraining)
SVM
Multi-layered Perceptron (Backpropagation)
XOR Problem
ADALINE
Perceptron
Electronic Brain
Golden Age
Dark Age ("AI Winter")
1943
1957
1960
1969
1986
1995
2006
1940
1950
1960
1970
1980
1990
2000
2010
S. McCulloch - W. Pitts
F. Rosenblatt
B. Widrow - M. Hoff
M. Minsky - S. Papert
D. Rumelhart - G. Hinton - R. Wiliams
V. Vapnik - C. Cortes
G. Hinton - S. Ruslan
X AND Y
X OR Y
NOT X
Foward Activity
Backward Error
• Adjustable Weights
• Weights are not Learned
• Learnable Weights and Threshold
• XOR Problem
• Solution to nonlinearly separable problems
• Big computation, local optima and overfitting
• Limitations of learning prior knowledge
• Kernel function: Human Intervention
• Hierarchical feature Learning

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

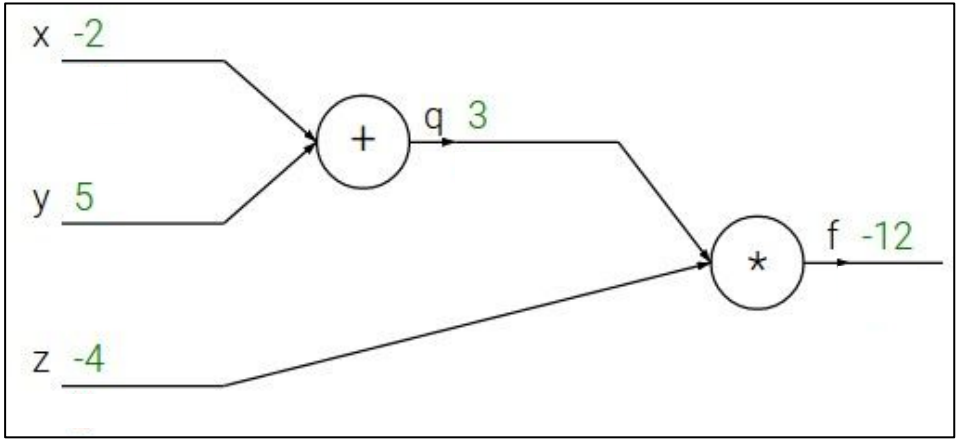


Queremos: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$



Queremos: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

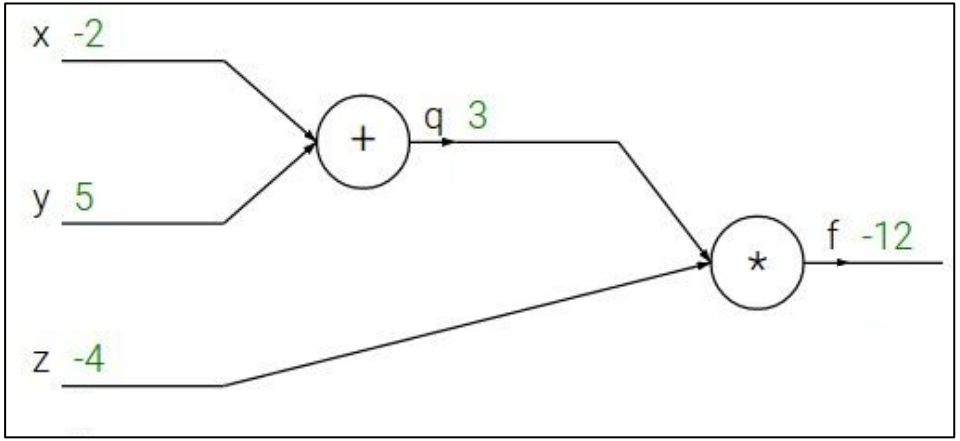
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Queremos: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



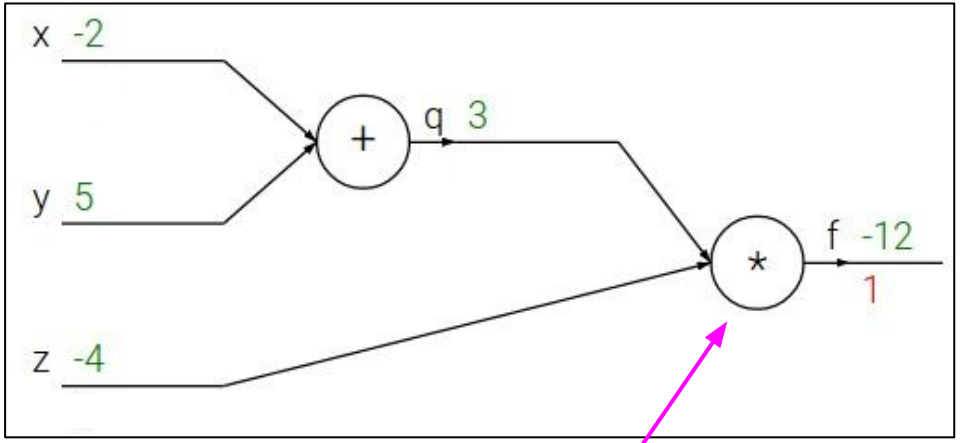
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Queremos: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{df}{df} = 1$$

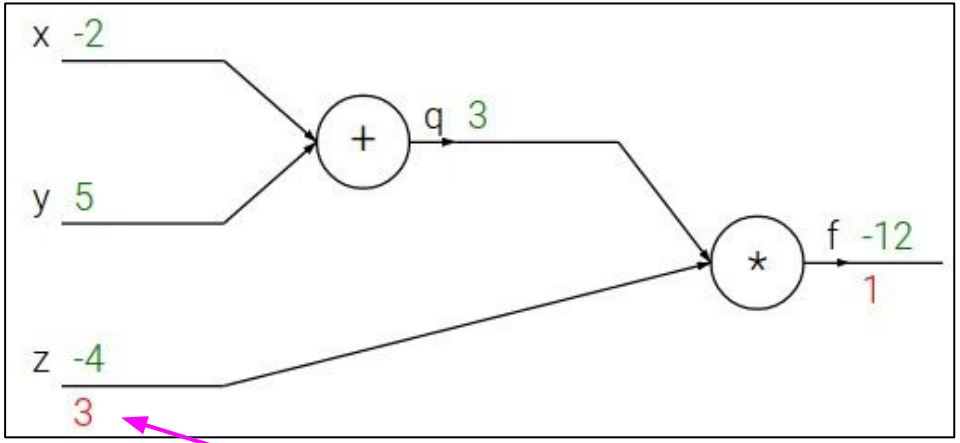
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Queremos: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{df}{dz} = q = 3$$

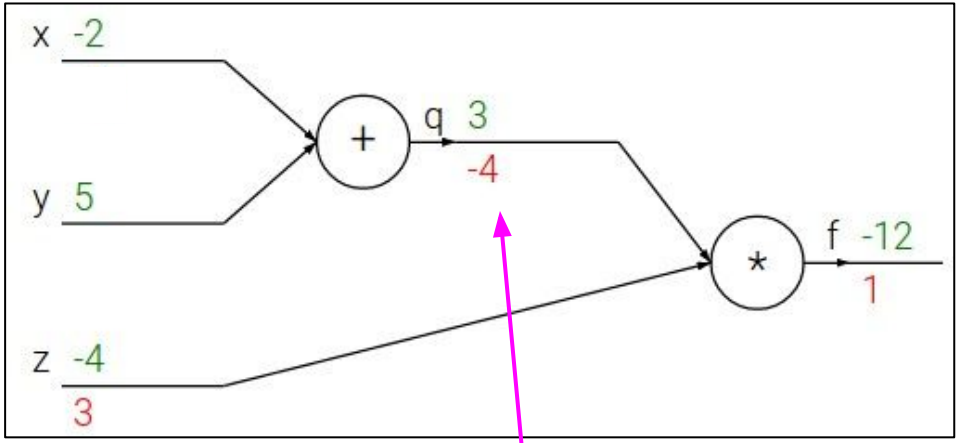
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Queremos: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{df}{dq} = z = -4$$

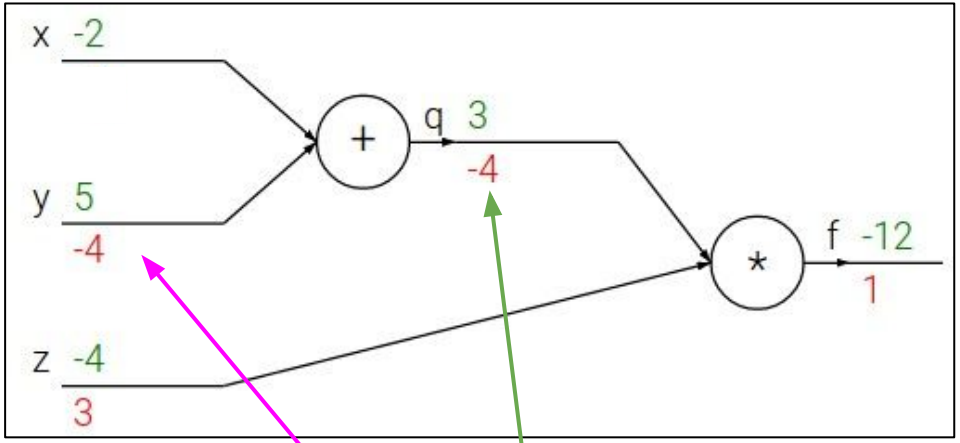
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Queremos: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{df}{dy} = \frac{df}{dq}\frac{dq}{dy} = -4 * 1 = -4$$

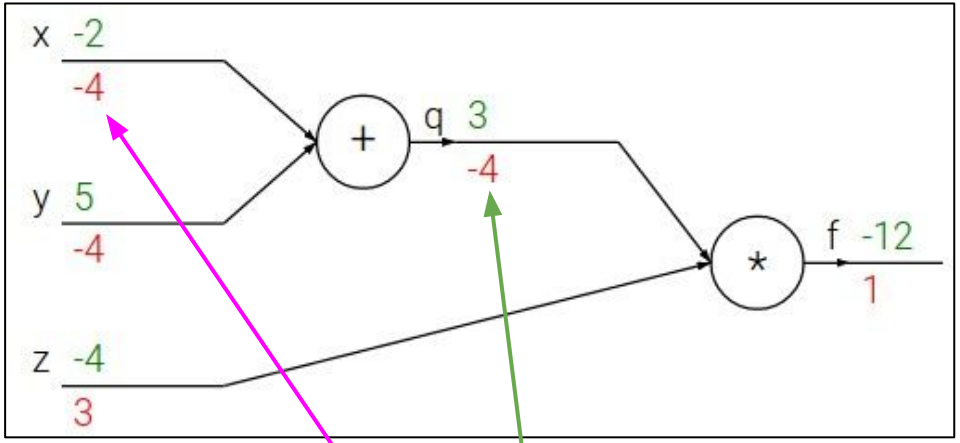
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

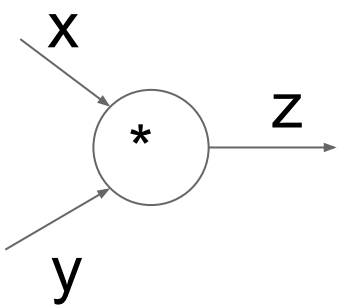
Queremos: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



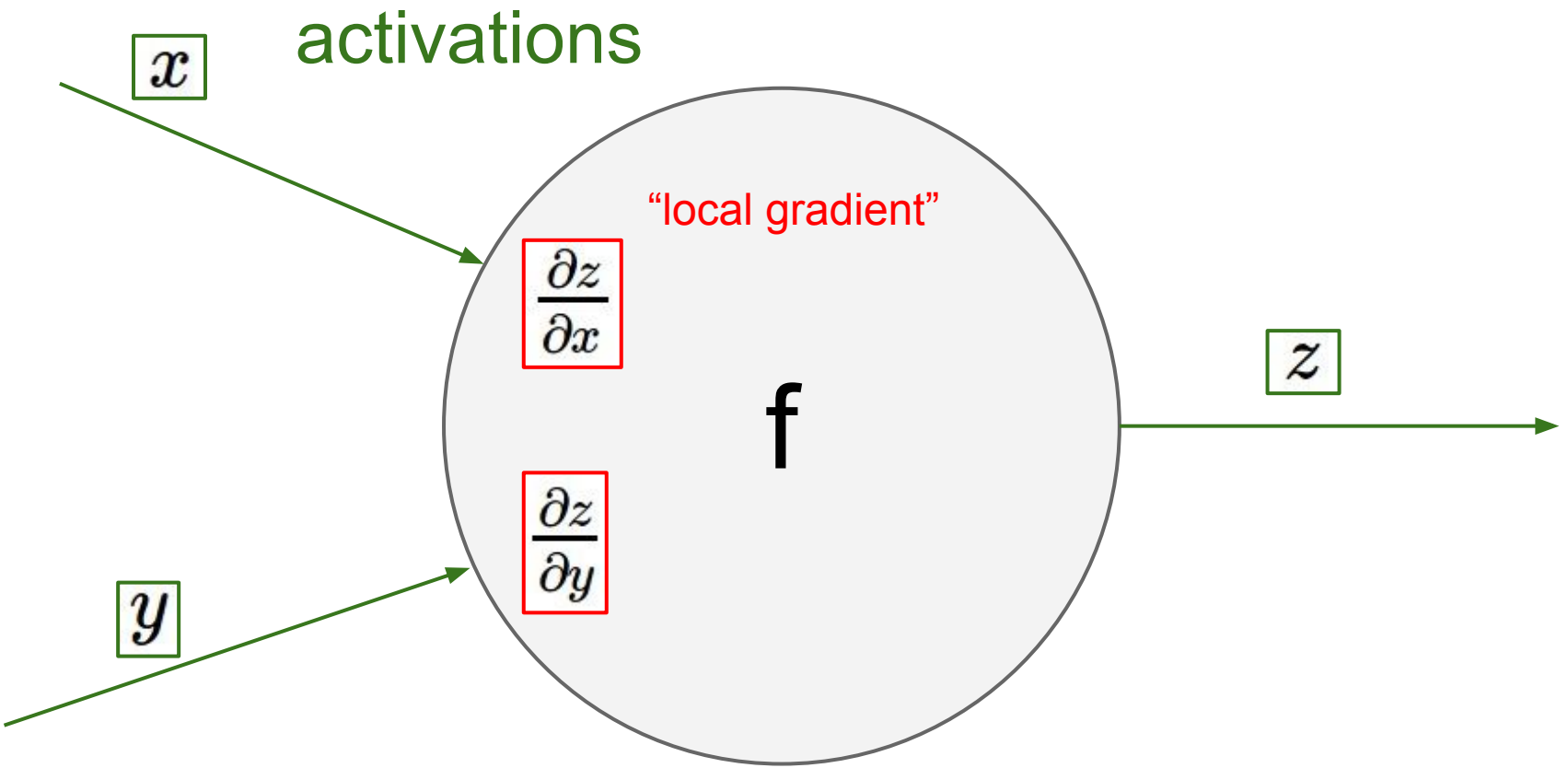
Chain rule:

$$\frac{df}{dx} = \frac{df}{dq}\frac{dq}{dx} = -4 * 1 = -4$$

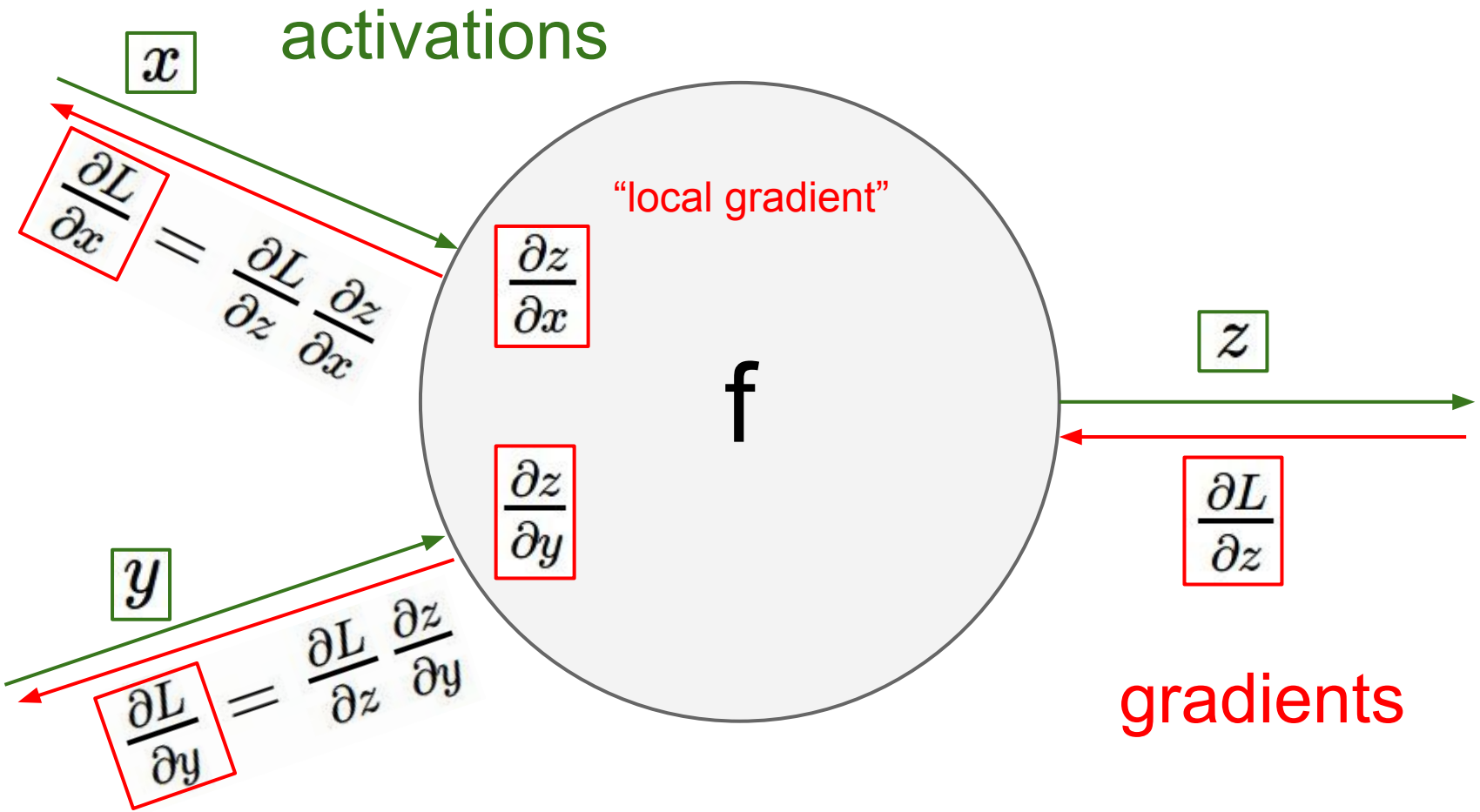
Implementation: forward/backward API




```
class MultiplyGate(object):
    def forward(x,y):
        z = x*y
        self.x = x # must keep these around!
        self.y = y
        return z
    def backward(dz):
        dx = self.y * dz # [dz/dx * dL/dz]
        dy = self.x * dz # [dz/dy * dL/dz]
        return [dx, dy]
```

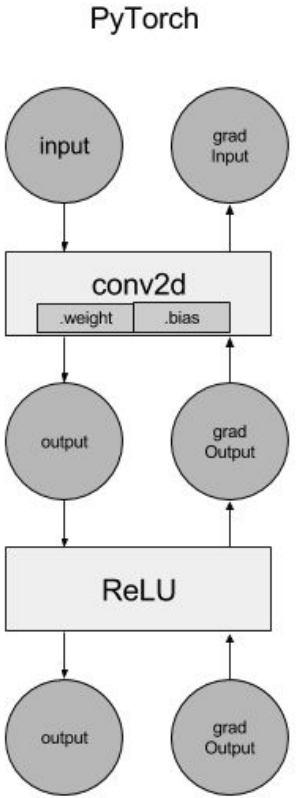


Forward pass



Backward pass

Ejemplo: PyTorch



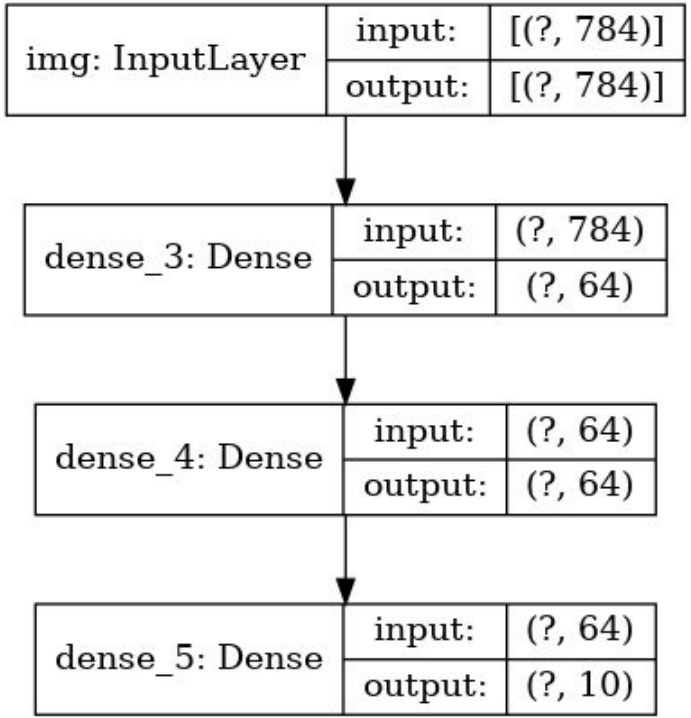
forward()

backward()

Intermediate states are held in the compute graph

Much easier to share weights (reuse the same module multiple times)

TensorFlow Keras



=



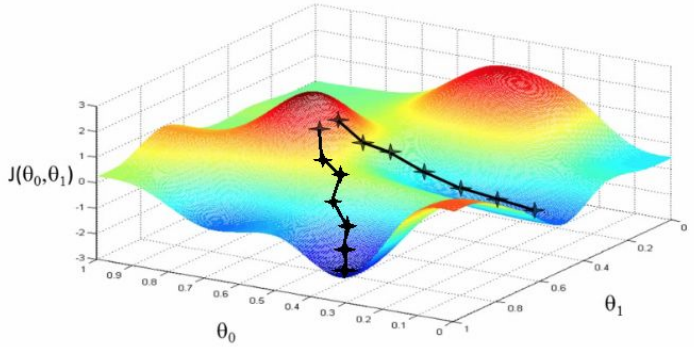




Optimización: Gradient descent

Iterar:

1. **Forward**: obtener la pérdida (Loss).
2. **Backprop:** calcular las gradientes.
3. **Actualizar** los parámetros del modelo.

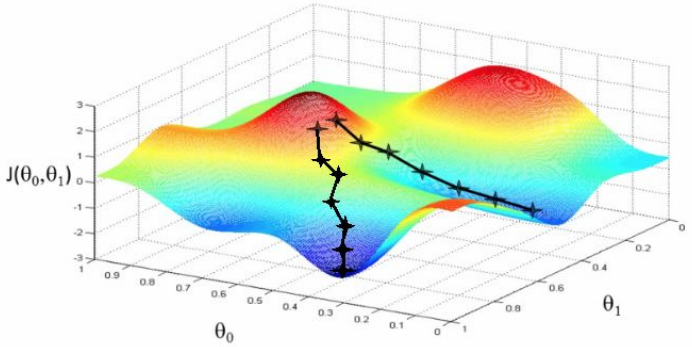


Optimización: Gradient descent

Iterar:

1. **Forward**: obtener la pérdida (Loss).
2. **Backprop:** calcular las gradientes.
3. **Actualizar** los parámetros del modelo.

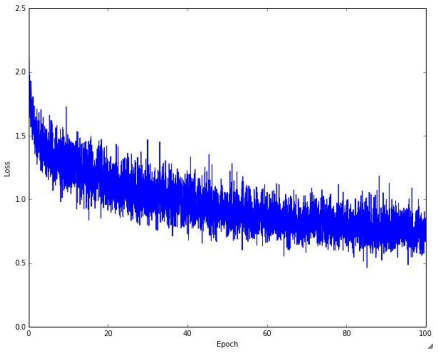
¿Y si tenemos mucha data?

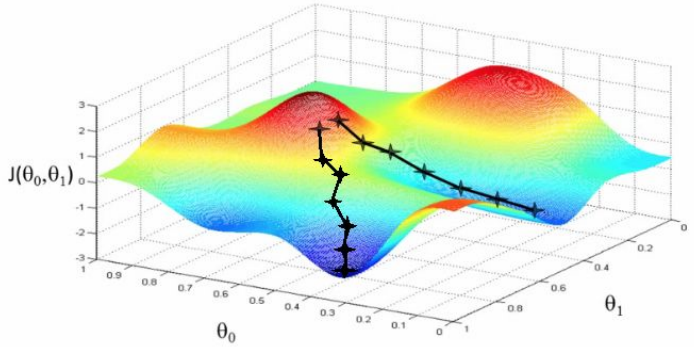


Optimización: Mini-batch Gradient descent

Iterar:

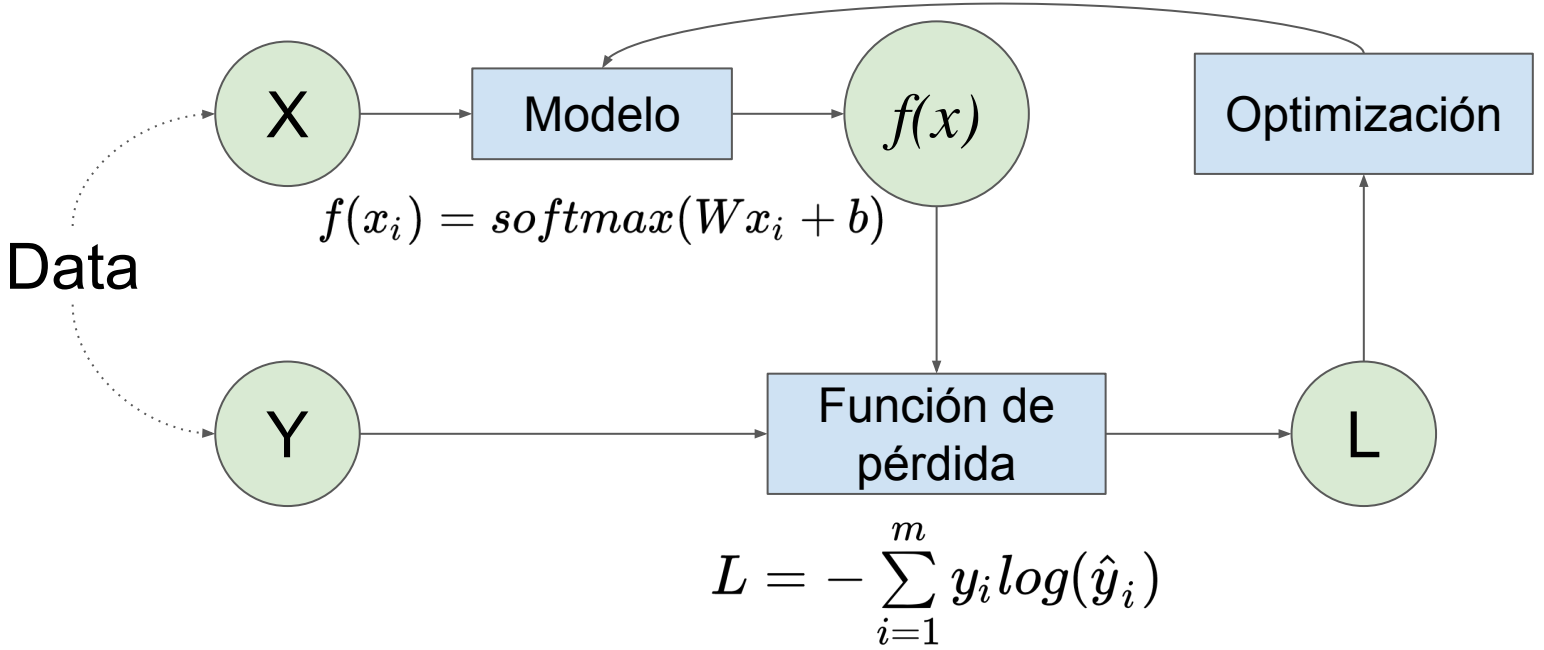
1. **Sample**: obtener una muestra de la data.
2. **Forward**: obtener la pérdida (Loss).
3. **Backprop:** calcular las gradientes.
4. **Actualizar** los parámetros del modelo.





Regresión logística

Backprop +
Mini-batch
Gradient descent



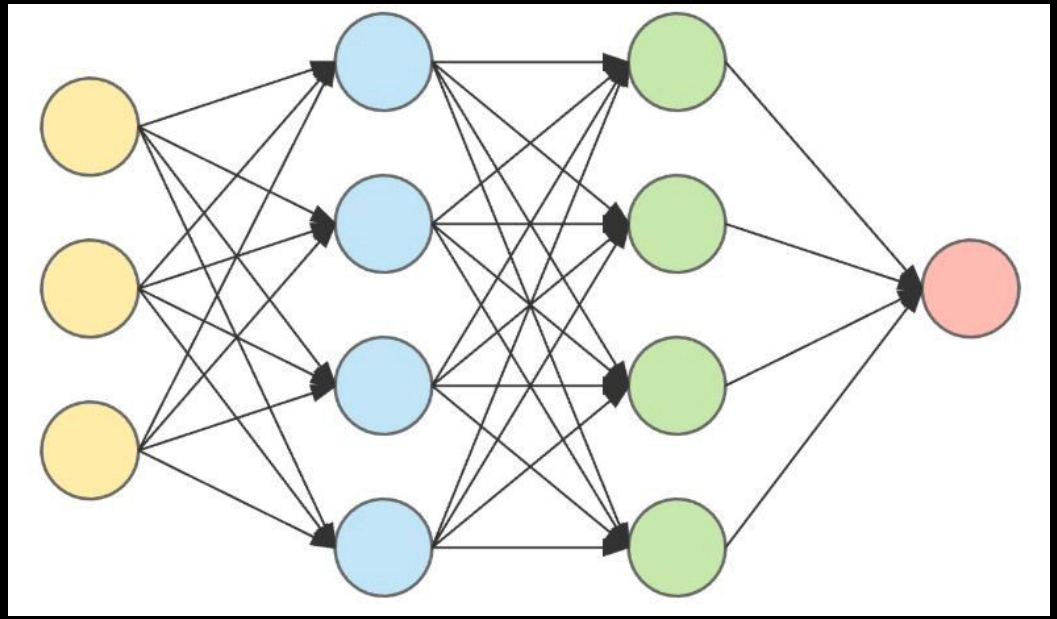
$$f(x_i) = softmax(Wx_i + b)$$

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i)$$

PUCP

MAESTRÍA EN
INFORMÁTICA

Deep Learning



Tema 3.4: Redes Neuronales

Prof. Dr. César A. Beltrán Castañón

Redes Neuronales

Hasta ahora teníamos funciones lineales antes de aplicar softmax:

$$f(x_i) = softmax(Wx_i + b)$$

¿Cómo incrementamos la complejidad del modelo?

Redes Neuronales

Hasta ahora teníamos funciones lineales antes de aplicar softmax:

$$f(x_i) = softmax(Wx_i + b)$$

¿Cómo incrementamos la complejidad del modelo?

Podemos añadir operaciones intermedias

Redes Neuronales

$$h(x_i) = W_1 x_i + b_1$$

10xN 10x4 4xN 10x1

Redes Neuronales

$$h(x_i) = W_1 x_i + b_1$$

10xN 10x4 4xN 10x1

$$f(x_i) = softmax(W_2 h(x_i) + b_2)$$

3xN 3x10 10xN 3x1

Redes Neuronales

$$h_1(x_i) = W_1 x_i + b_1$$

$$h_2(x_i) = W_2 h_1(x_i) + b_2$$

$$\vdots$$

$$f(x_i) = softmax(W_m h_n(x_i) + b_m)$$

Redes Neuronales

$$h_1(x_i) = W_1 x_i + b_1$$

$$h_2(x_i) = W_2 h_1(x_i) + b_2$$

$$\vdots$$

$$f(x_i) = softmax(W_m h_n(x_i) + b_m)$$

Pero una serie de operaciones lineales se puede reducir a solo 1.

$$y_1 = 2x + 10$$

$$y_2 = 3y_1 - 5$$

$$y_2 = 6x + 25$$

Redes Neuronales

$$h_1(x_i) = \sigma(W_1 x_i + b_1)$$

$$h_2(x_i) = \sigma(W_2 h_1(x_i) + b_2)$$

.

.

.

$$f(x_i) = softmax(W_m h_n(x_i) + b_m)$$

Pero una serie de operaciones lineales se puede reducir a solo 1.

Añadimos funciones de activación

Redes Neuronales

Pero una serie de operaciones lineales se puede reducir a solo 1.

$$h_1(x_i) = \sigma(W_1 x_i + b_1)$$

$$h_2(x_i) = \sigma(W_2 h_1(x_i) + b_2)$$

Añadimos funciones de activación

.

.

.

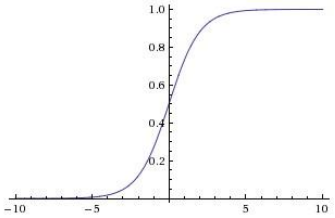
$$f(x_i) = softmax(W_m h_n(x_i) + b_m)$$

softmax también es una función de activación

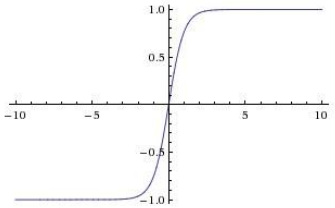
Funciones de Activación

Sigmoid

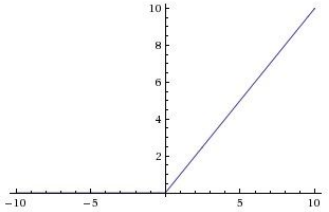
$$\sigma(x) = 1/(1 + e^{-x})$$



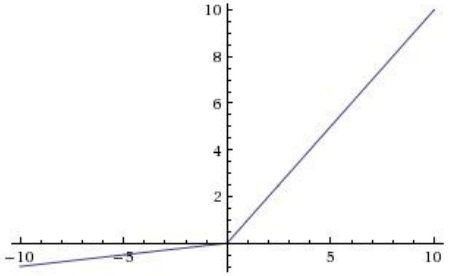
tanh tanh(x)



ReLU max(0,x)

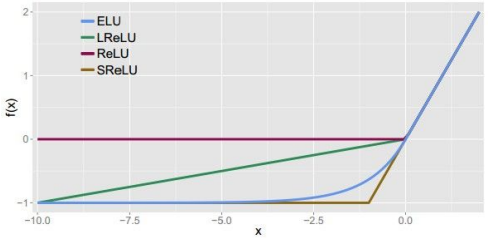


Leaky ReLU
max(0.1x, x)



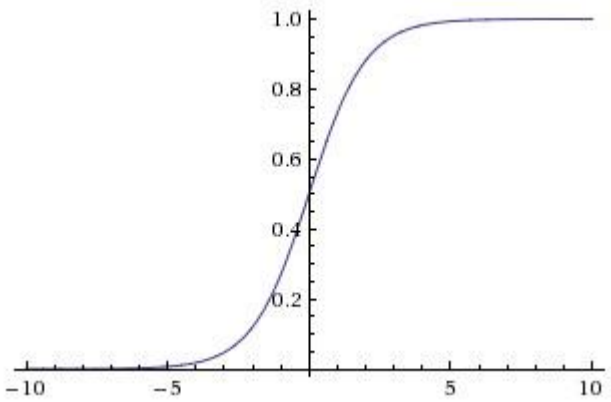
ELU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \le 0 \end{cases}$$



Funciones de Activación

$$\sigma(x) = 1/(1 + e^{-x})$$

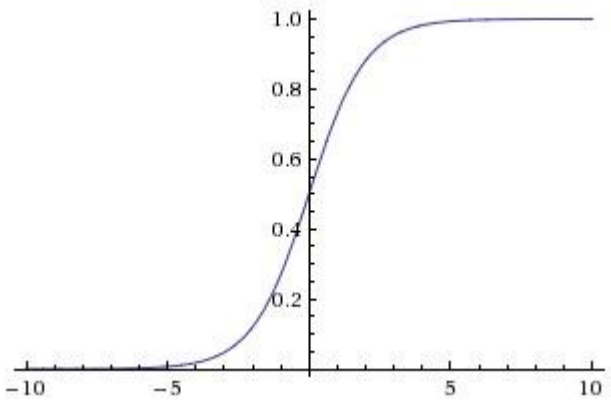


Sigmoid

- Reduce el rango a [0,1]
- Históricamente popular, por la interpretación de “activar” una neurona.

Funciones de Activación

$$\sigma(x) = 1/(1 + e^{-x})$$

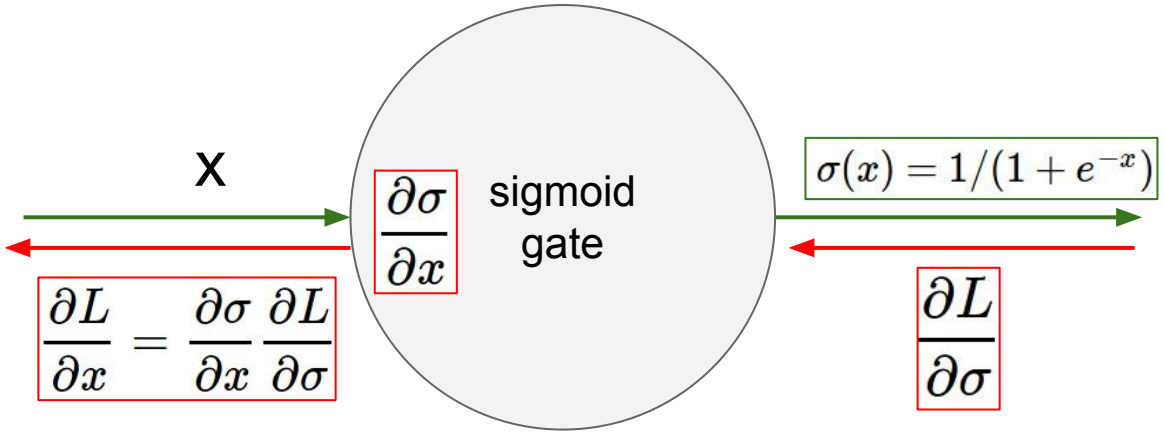


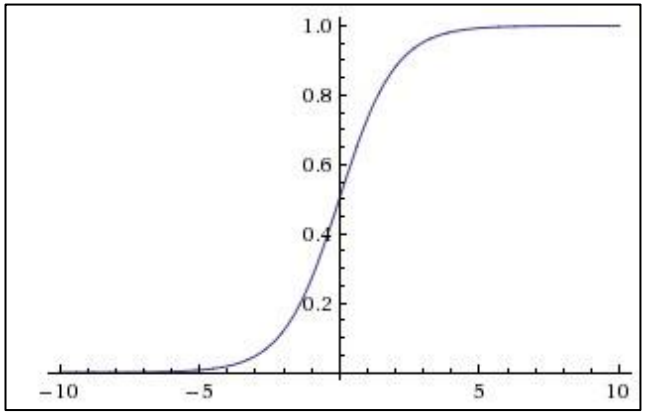
Sigmoid

- Reduce el rango a [0,1]
- Históricamente popular, por la interpretación de “activar” una neurona.

3 problemas:

1. Las neuronas saturadas “matan” las gradientes.

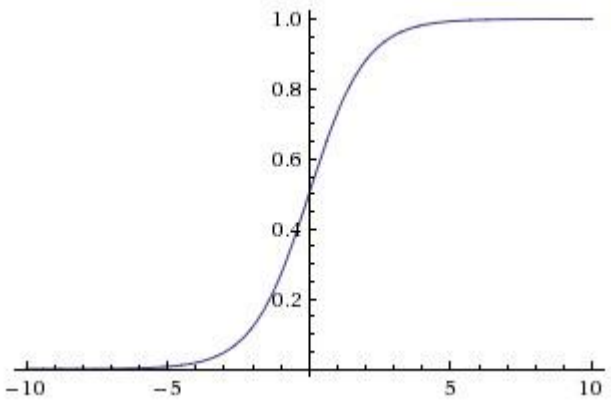




¿Qué pasa cuando x = -10?
¿Qué pasa cuando x = 10?

Funciones de Activación

$$\sigma(x) = 1/(1 + e^{-x})$$



Sigmoid

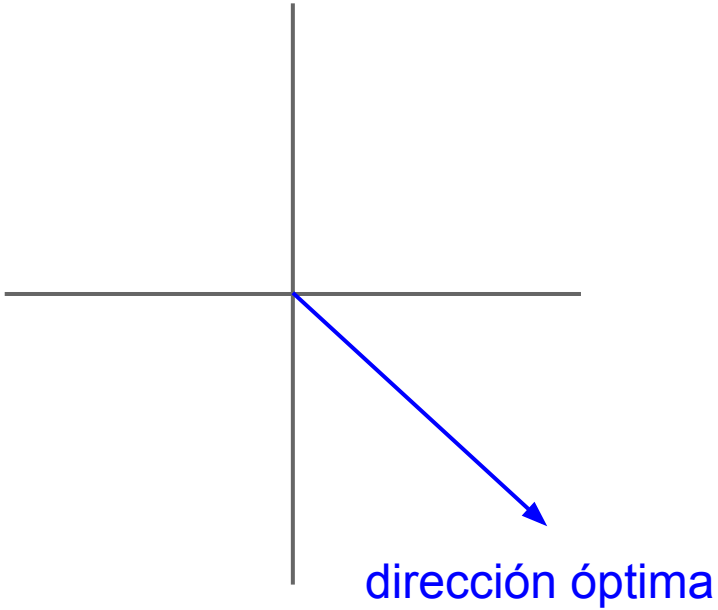
- Reduce el rango a [0,1]
- Históricamente popular, por la interpretación de “activar” una neurona.

3 problemas:

1. Las neuronas saturadas “matan” las gradientes.
2. El outputs no está centrado en cero.

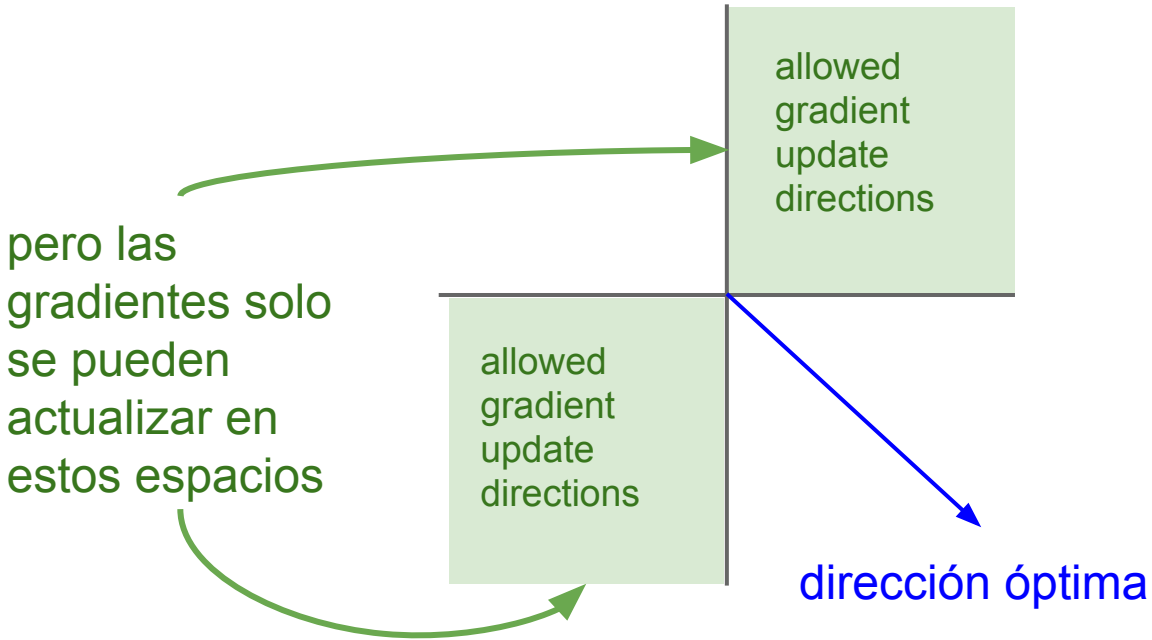
Cuando el input (x) de una neurona siempre es positivo, las gradientes van a ser todas positivas o todas negativas.

Espacio de optimización de la gradiente:



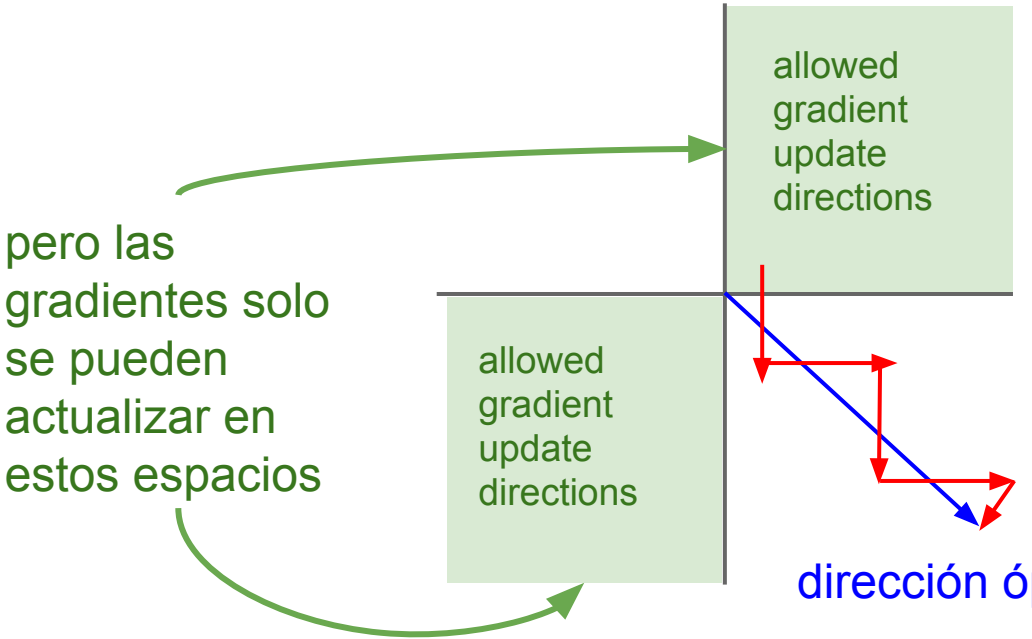
Cuando el input (x) de una neurona siempre es positivo, las gradientes van a ser todas positivas o todas negativas.

Espacio de optimización de la gradiente:



Cuando el input (x) de una neurona siempre es positivo, las gradientes van a ser todas positivas o todas negativas.

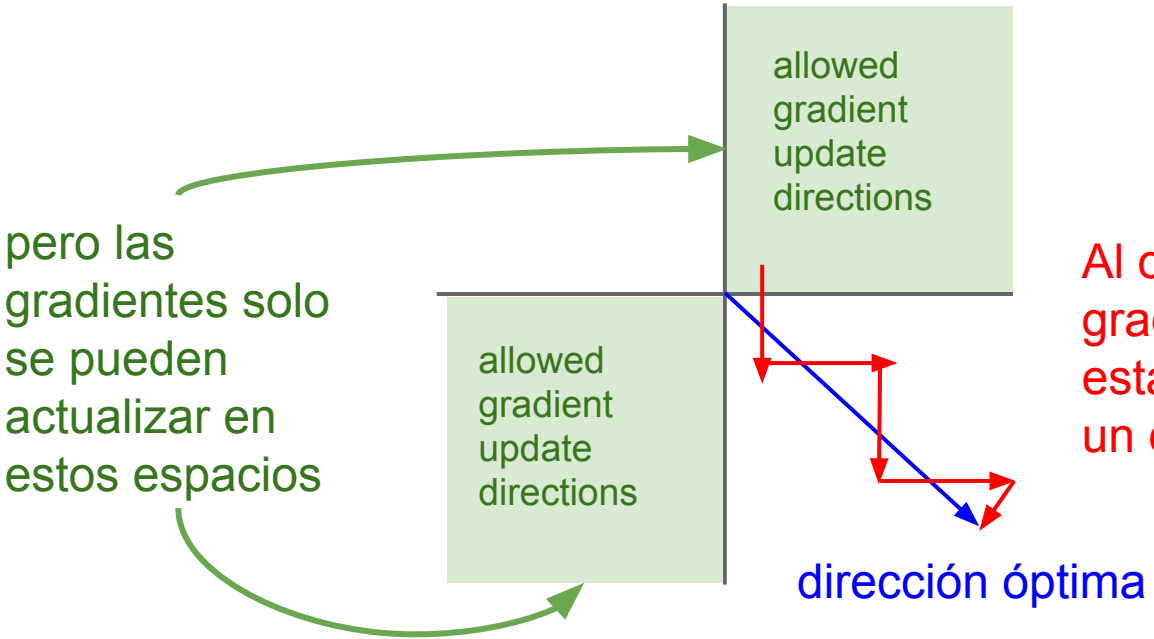
Espacio de optimización de la gradiente:



Al optimizar usando gradient descent, vamos a estar obligados a seguir un camino en zig zag

Cuando el input (x) de una neurona siempre es positivo, las gradientes van a ser todas positivas o todas negativas.

Espacio de optimización de la gradiente:

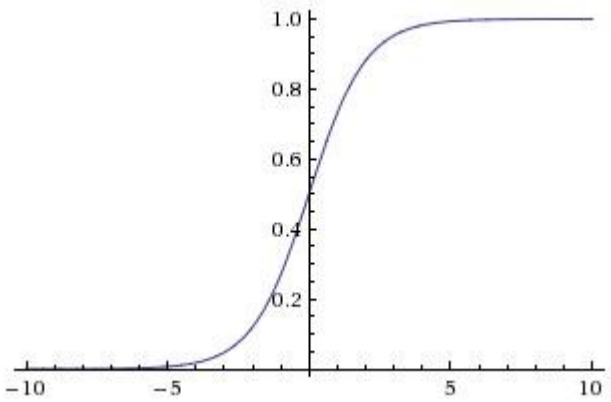


Al optimizar usando gradient descent, vamos a estar obligados a seguir un camino en zig zag

(este es uno de los motivos por lo que normalizamos la data)

Funciones de Activación

$$\sigma(x) = 1/(1 + e^{-x})$$



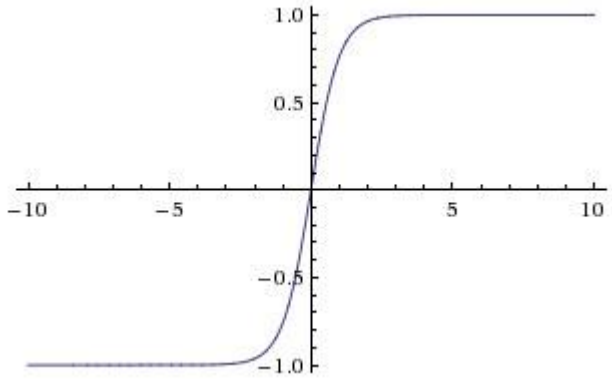
Sigmoid

- Reduce el rango a [0,1]
- Históricamente popular, por la interpretación de “activar” una neurona.

3 problemas:

1. Las neuronas saturadas “matan” las gradientes.
2. El outputs no está centrado en cero.
3. La función exponencial es pesada de calcular.

Funciones de Activación

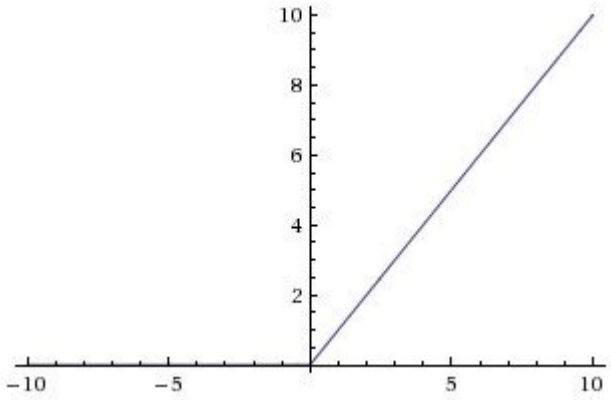


- Reduce el rango a [-1,1]
- Centrada en cero :D
- Neuronas saturadas :(

$$\tanh(x) = \frac{(e^{x} - e^{-x})}{(e^{x} + e^{-x})}$$

Funciones de Activación

relu(x) = max(0,x)



- No se satura (en los positivos).
- Computacionalmente eficiente.

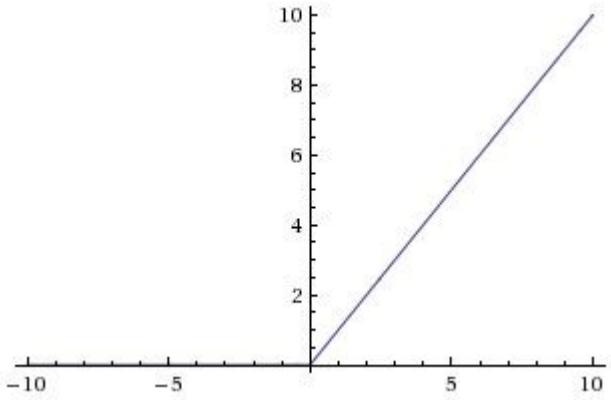
- No está centrada en cero.
- Las gradientes cuando x<0 es ?

ReLU
(Rectified Linear Unit)

[Krizhevsky et al., 2012]

Funciones de Activación

relu(x) = max(0,x)



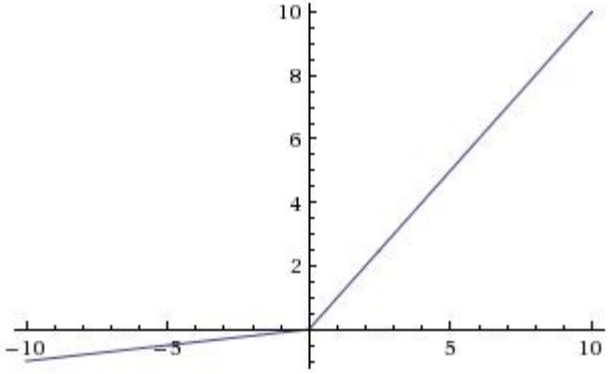
- No se satura (en los positivos).
- Computacionalmente eficiente.

- No está centrada en cero.
- Las gradientes cuando x<0 es 0.

ReLU
(Rectified Linear Unit)

[Krizhevsky et al., 2012]

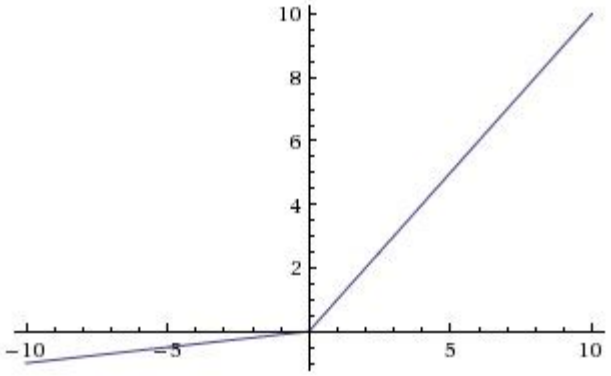
Funciones de Activación



Leaky ReLU

$$f(x) = \max(0.01x, x)$$

Funciones de Activación



Leaky ReLU

$$f(x) = \max(0.01x, x)$$

Parametric Rectifier (PReLU)

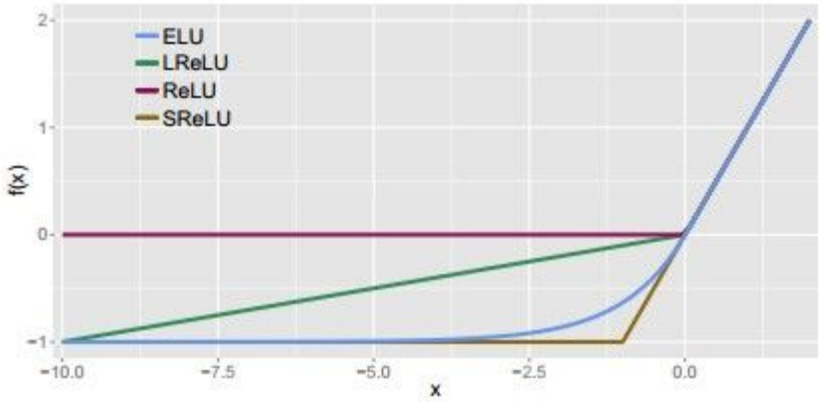
$$f(x) = \max(\alpha x, x)$$

alpha es un parámetro a optimizar

[Mass et al., 2013]
[He et al., 2015]

Funciones de Activación

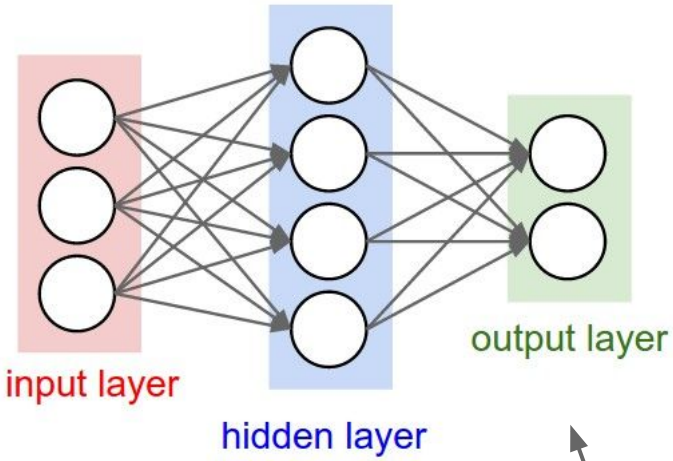
Exponential Linear Units (ELU)

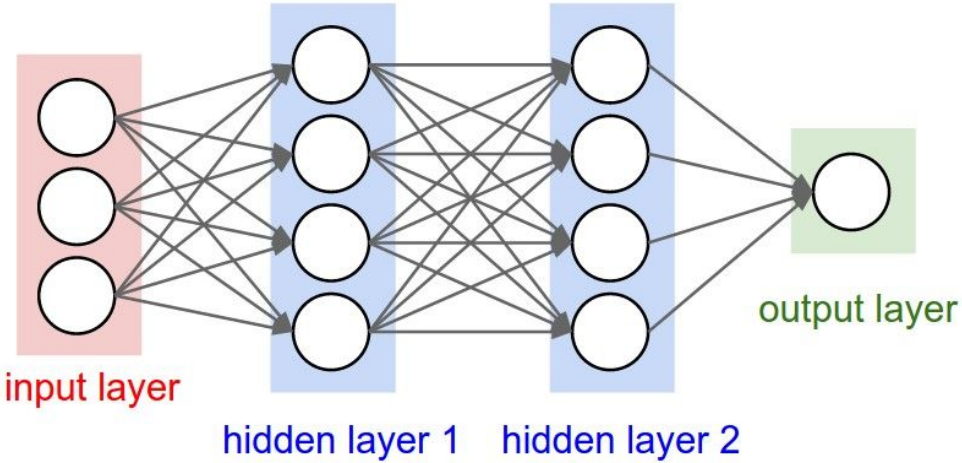


$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\left(\exp(x) - 1\right) & \text{if } x \leq 0 \end{cases}$$

[Clevert et al., 2015]

Redes Neuronales



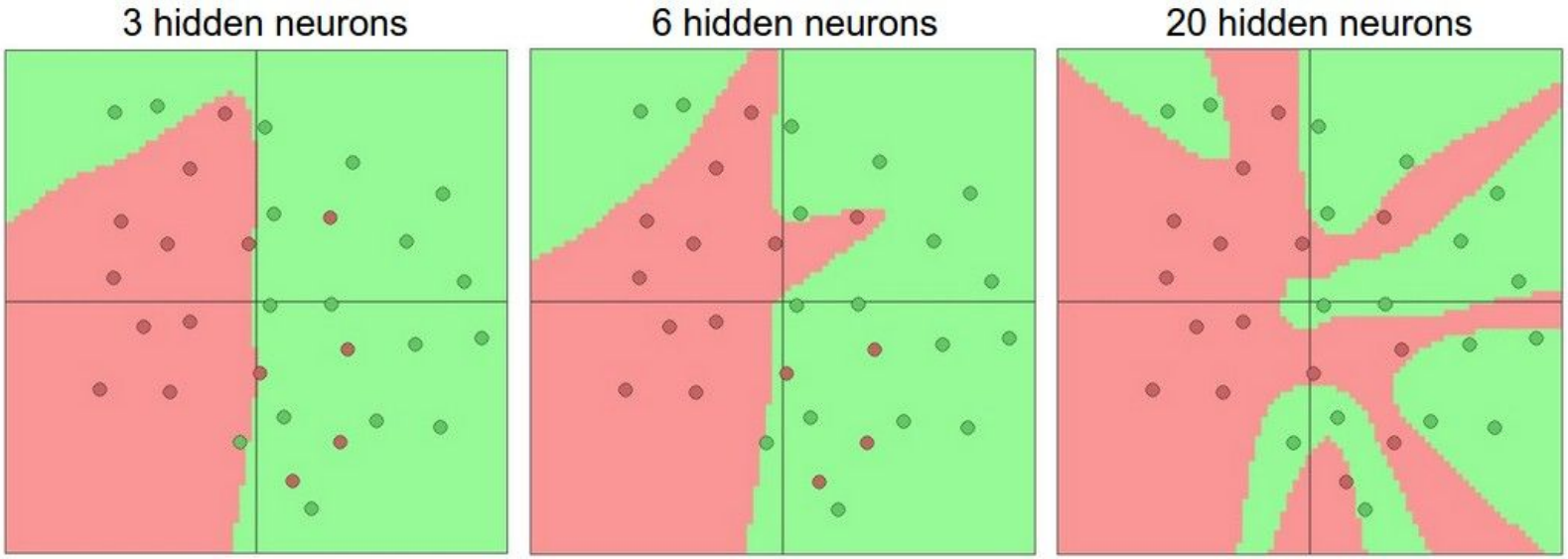


“2-layer Neural Net”, or
“1-hidden-layer Neural Net”

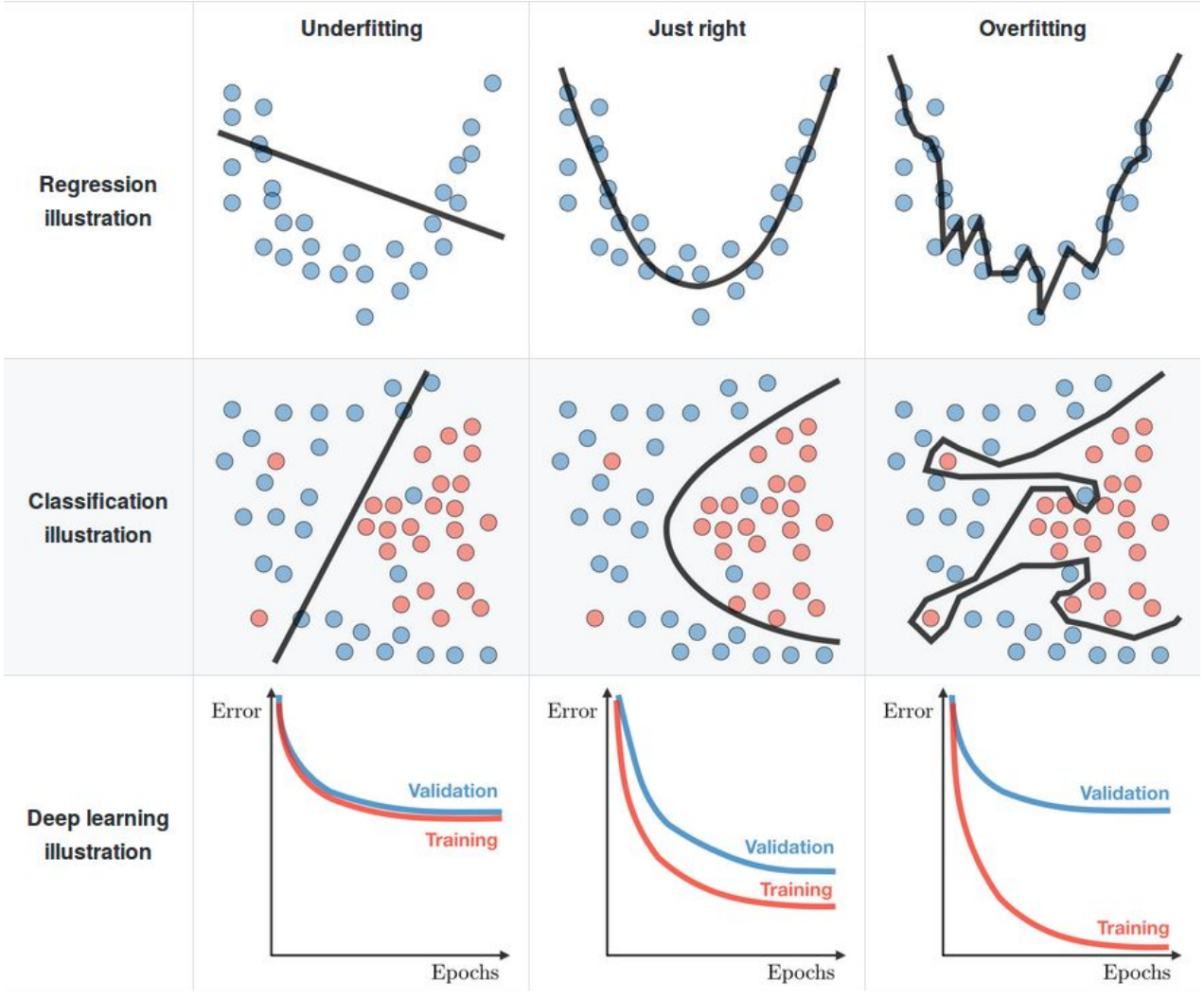
“3-layer Neural Net”, or
“2-hidden-layer Neural Net”

“Fully-connected” layers

¿Número de capas, de neuronas?



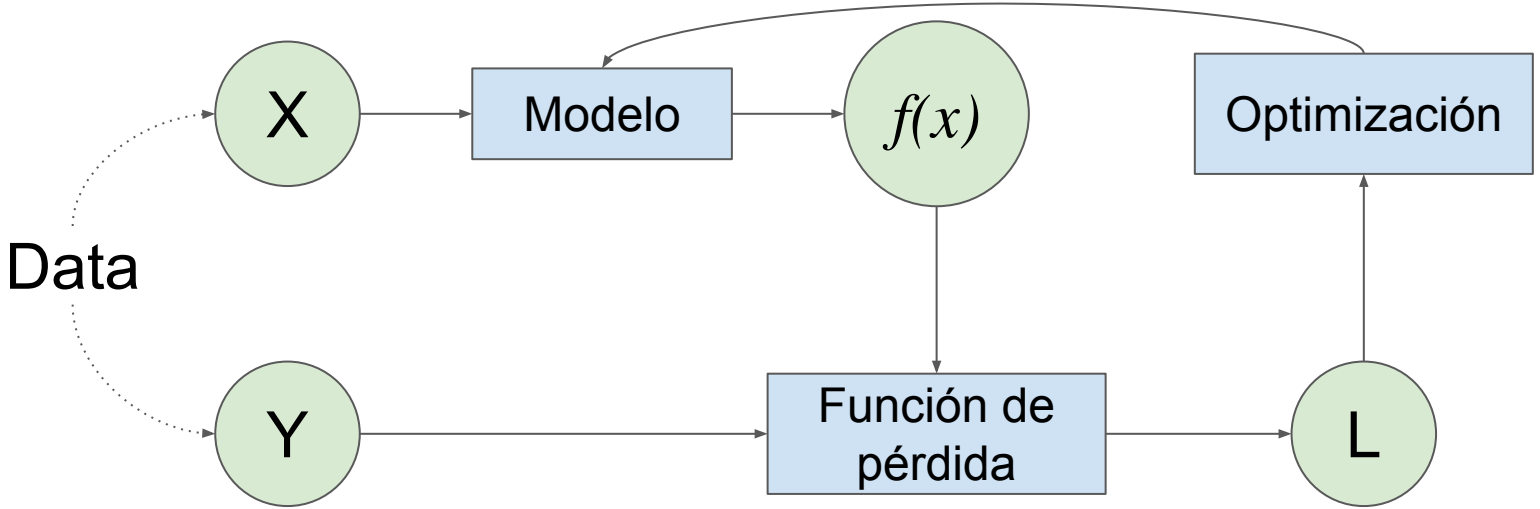
de neuronas?.... *hiperparametros*


Underfitting
Just right
Overfitting
Regression illustration
Classification illustration
Deep learning illustration
Error
Validation
Training
Epochs
Error
Validation
Training
Epochs
Error
Validation
Training
Epochs

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Possible remedies	• Complexify model • Add more features • Train longer		• Perform regularization • Get more data

Redes Neuronales

Backprop +
Mini-batch
Gradient descent



Calcule la salida de las siguientes redes:

